“Linux System Debugging Tool”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

DHARMIKA	**2520030012**
ANUSHKA K	**2520030501**
RUSHVINI	**2520030542**

Under the Esteemed Guidance of

Dr. K. Anusha
Assistant Professor
Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **"Linux System Debugging Tool"** is a record of Bonafide work of **Dharmika - 2520030012, Anushka K - 2520030501, Rushvini - 2520030542** submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

DHARMIKA	**2520030012**
ANUSHKA K	**2520030501**
RUSHVINI	**2520030542**

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is to certify that the mini project based report entitled **"Linux System Debugging Tool"** is a bonafide work done and submitted by **Dharmika - 2520030012, Anushka K - 2520030501, Rushvini - 2520030542** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2026-2027**.

Signature of the Guide

Signature of the Course Coordinator **Signature of the HOD**

ACKNOWLEDGEMENT

The success of this project would not have been possible without the timely help and guidance rendered by many people. We wish to express our sincere thanks to all those who have assisted us in one way or the other in the completion of our project.

Our greatest appreciation goes to our Course Coordinator Dr. K. Anusha and our guide Dr. K. Anusha, Department of Computer Science and Engineering, whose tremendous support, encouragement and guidance for this project cannot be expressed in words.

We express our gratitude to **Dr. P Venkateshwara Rao (CSE)**, Head of the ***Department of Computer Science and Engineering***, for providing us with adequate facilities, ways and means by which we were able to complete this project-based Lab.

We thank all the members of the teaching and non-teaching staff, and also those who have assisted us directly or indirectly in the successful completion of this project. Finally, we sincerely thank our parents, friends and classmates for their kind help and cooperation during our work.

DHARMIKA **2520030012**

ANUSHKA K **2520030501**

RUSHVINI **2520030542**

Abstract

Linux powers servers, cloud platforms, smartphones and embedded devices, yet diagnosing a misbehaving Linux program remains difficult. Developers must combine many separate utilities such as ps, lsof, pmap, strace and gdb, each with its own syntax and output, and most of them are large programs whose internal working is hidden from the learner. This fragmentation slows down debugging and prevents students from seeing how the Operating System features they study in theory actually work together.

This project presents the **Linux System Debugging Tool (LSDT)**, a compact, dependency-free command-line debugger written in C that obtains all of its information directly from the kernel. LSDT combines eight modules in one binary: a **process inspector**, a **file-descriptor inspector**, a **thread inspector**, a **memory-map analyser**, a **live resource monitor**, a **system-call tracer**, a **crash analyser** and a **system overview**. They are built on the /proc filesystem, the ptrace system call, signals, pipes and POSIX threads. The system-call tracer launches a program under ptrace and reports every call with its arguments, result, error name and duration. The crash analyser reports the signal, its cause code, the fault address, the CPU registers and the memory region of the faulting instruction. The live monitor is a producer-consumer pair of threads connected by a pipe and synchronised with a mutex.

The tool is about 600 lines long, uses no dynamic memory allocation and compiles without warnings with GCC on Ubuntu 24.04 (native or WSL 2). It was validated with purpose-built test programs: a NULL-pointer dereference, a division by zero and a multi-threaded CPU and memory consumer. All 14 test cases passed, the instruction pointers reported by the crash analyser matched the disassembly produced by objdump exactly, and tracing overhead was measured at about six times for a system-call-intensive command. In summary, LSDT is a practical diagnostic utility and a transparent teaching aid that demonstrates processes, threads, virtual memory, signals, file descriptors, scheduling, inter-process communication and synchronisation in one real program.

Keywords: Linux, debugging, /proc, ptrace, system calls, signals, memory map, POSIX threads, Ubuntu, WSL.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		**i**
	Abstract		**ii**
	Index		**iii**
1	**Introduction**		**1–5**
		1.1 Background of the Project	1
		1.2 Problem Statement	2
		1.3 Scope of the Project	2
		1.4 Objectives of the Project	4
		1.5 Importance of the Application	4
		1.6 Target Users / Audience	5
2	**OS Concepts and Background Study**		**6–12**
		2.1 Operating System Concepts Related to the Project	6
		2.2 Existing Methods / Algorithms	10
		2.3 Proposed Approach	11
		2.4 Advantages of the Proposed Approach	12
3	**System Requirements**		**13–15**
		3.1 Hardware Requirements	13
		3.2 Software Requirements	13
		3.3 Operating System Environment - Ubuntu / WSL	14
		3.4 Development Tools and Compilers	14
4	**Technology and Tools Used**		**16–20**
		4.1 C / C++ Programming	16
		4.2 GCC Compiler	16
		4.3 Linux System Calls	17
		4.4 POSIX Threads / Synchronization	18
		4.5 Linux Commands and Utilities	19
		4.6 Other Tools / Libraries Used	20
5	**System Architecture and Design**		**21–29**
		5.1 Overall System Architecture	21
		5.2 System Workflow	22
		5.3 Flowchart / Data Flow	24
		5.4 Module Description	27
		5.5 Input and Output Design	28
6	**Implementation**		**30–48**
		6.1 Module-wise Implementation	30
		6.2 Algorithm / Pseudocode	32
		6.3 Core C / C++ Implementation	34
		6.4 Linux System Calls / APIs Used	42
		6.5 Process / Thread Implementation	44
		6.6 IPC / Synchronization Implementation	44

S. No.	Chapter	Topics / Subtopics	Page No.
		6.7 Memory Management Implementation	46
		6.8 Optional User Interface Implementation	47
7	**Execution and Demonstration**		**49–59**
		7.1 Compilation in Ubuntu	49
		7.2 Execution Commands	50
		7.3 Sample Inputs and Outputs	50
		7.4 Screenshots of Execution	51
		7.5 Demonstration of OS Concepts	58
8	**Conclusion**		**60–62**
		8.1 Summary of the Project	60
		8.2 Objectives Achieved	60
		8.3 Results Achieved	61
		8.4 Skills and Learning Outcomes	61
		8.5 Limitations and Future Scope	61
9	**References**		**63–64**
		9.1 Books	63
		9.2 Research Papers	63
		9.3 Linux / GNU Documentation	63
		9.4 Online Resources	64

CHAPTER - 1
INTRODUCTION

1.1 Background of the Project

Linux is the most widely deployed operating system kernel in the world. It runs the servers that host websites and cloud platforms, the supercomputers used for scientific research, the Android phones in our pockets, the routers in our homes and the embedded controllers inside cars and televisions. Because so many critical systems depend on Linux, the ability to understand what a Linux system is doing at any moment, and to find out why a program misbehaves, is one of the most valuable skills a computer engineer can have.

When a program on Linux fails, the visible symptom is usually small: a window that does not open, a command that prints "Segmentation fault", a server that becomes slow, or a file that cannot be found. The real cause, however, lies deep inside the interaction between the program and the operating system: a pointer that refers to memory that was never mapped, a system call that returned an error nobody checked, a thread that keeps consuming the processor, or a file descriptor that was never closed. Finding such causes is called **debugging**, and doing it well requires visibility into the operating system itself.

Linux provides this visibility through a small number of well-defined kernel interfaces. The **/proc pseudo-filesystem** exposes the live state of every process and of the whole machine as ordinary text files. The **ptrace system call** allows one process to observe and control another process, intercept its system calls and examine its CPU registers. **Signals** inform a process of exceptional events, such as an illegal memory access or a division by zero. **Pipes and threads** allow a tool to collect measurements concurrently while it presents results. Professional tools such as `strace`, `gdb`, `lsof`, `pmap`, `top` and `valgrind` are all built on top of these same interfaces.

In a typical Operating Systems course, these interfaces are studied separately and mostly in theory: students learn that a process has a state, that virtual memory is divided into segments, and that signals can terminate a program, but they rarely see these ideas working together inside one real program. This project, the **Linux System Debugging Tool (LSDT)**, closes that gap. LSDT is a command-line debugging utility written entirely in C that talks directly to the kernel through /proc, ptrace, signals, pipes and POSIX threads. It consolidates eight diagnostic capabilities (process inspection, open-file inspection, thread inspection, memory-map analysis, live resource monitoring, system-call tracing, crash analysis and system overview) into a single, small, easy-to-read program.

The tool was developed and tested on Ubuntu 24.04 LTS and is designed to run equally well on a native Ubuntu installation or on Ubuntu running inside the Windows Subsystem for Linux (WSL 2). It does not depend on any third-party library: it needs only the GNU C compiler and the C library that ship with every Linux distribution.

1.1.1 What is debugging?

Debugging is the systematic process of locating and removing the cause of incorrect behaviour in software. For operating-system level problems, four complementary activities are normally used, and LSDT supports each of them:

Activity	Question it answers	Kernel interface used	LSDT command
Static inspection	What is this process? Who started it? What resources does it hold?	/proc/PID/status, cmdline, exe, cwd, limits, fd	`info`, `fds`, `threads`, `sys`
Memory analysis	How is the address space laid out? How much memory is really used?	/proc/PID/maps, smaps_rollup	`mem`
Dynamic tracing	Which system calls does the program make, with what arguments and results?	ptrace(PTRACE_SYSCALL), waitpid	`trace`, `trace -c`
Post-mortem / crash analysis	Why did the program die? Which instruction and address caused it?	ptrace(GETSIGINFO, GETREGS), signals	`crash`
Resource profiling	How much CPU and memory is the program using over time?	/proc/PID/stat, pipe, pthreads	`watch`

1.2 Problem Statement

Debugging on Linux today is a fragmented experience. A student or developer who wants to investigate a misbehaving program must learn and combine many unrelated utilities: `ps` or `top` for process state, `lsof` for open files, `pmap` for memory maps, `strace` for system calls, `gdb` for crashes, and `cat /proc/...` for everything else. Each tool has its own command-line syntax, its own output format and its own learning curve. Several of them are large programs whose internal working is hidden from the user, so the learner uses the tool as a black box and never understands which operating-system mechanism produced the answer.

The problem addressed by this project can therefore be stated as follows:

"To design and develop a lightweight, single-binary Linux System Debugging Tool in C that uses the kernel's own interfaces (/proc, ptrace, signals, pipes and POSIX threads) to inspect running processes, trace system calls, analyse process memory, monitor resource usage in real time and report the exact cause of a program crash, while keeping the source code small and readable enough to demonstrate the underlying Operating System concepts."

More specifically, the following concrete difficulties motivate the work:

- **Tool fragmentation:** at least six separate commands are needed to obtain a full picture of one process.
- **Opaque implementation:** production tools such as gdb and strace consist of hundreds of thousands of lines of code; they cannot be studied in a semester.
- **Cryptic crash messages:** a plain "Segmentation fault (core dumped)" message does not tell the programmer which address was accessed, which instruction faulted or whether the address was mapped at all.
- **Lack of integrated measurement:** the effect of a bug on CPU use, memory growth and thread count is normally observed in a different window from the cause.
- **Educational gap:** students do not usually see ptrace, /proc, signals, pipes and mutexes working together in one real, useful program.

1.3 Scope of the Project

The scope defines what the tool does and, equally important, what it intentionally does not do.

1.3.1 In scope

- A command-line tool for 64-bit Linux (x86_64) written in the C language and compiled with GCC.
- **Process inspection:** name, state, parent, user IDs, thread count, virtual and resident memory, context switches, command line, executable path, working directory and open-file limit of any process whose PID is given.
- **File-descriptor inspection:** listing of every open descriptor with its type (regular file, directory, pipe, socket, character device) and target.
- **Thread inspection:** thread IDs, names, scheduling state and user/kernel CPU time for all threads of a process.
- **Memory-map analysis:** classification of every mapping into heap, stack, file-backed, anonymous and special regions, with totals and a summary of resident (RSS), proportional (PSS) and private-dirty memory.
- **System-call tracing:** launching a program under ptrace, printing each system call with its arguments, return value, error name and duration, and printing a per-call statistics table.
- **Crash analysis:** running a program under supervision and, when it receives a fatal signal, reporting the signal, its cause code, the faulting address, all key CPU registers and the memory region in which the fault happened.
- **Live resource monitoring:** periodic sampling of CPU percentage, resident memory and thread count using a producer thread, a pipe and a consumer thread synchronised with a mutex, with results logged to a file.
- **System overview:** load average, uptime, memory totals and number of running processes.

1.3.2 Out of scope

- A graphical user interface (the tool is text based; a terminal UI is listed as future work).
- Source-level debugging such as breakpoints, single-stepping, symbol resolution and DWARF debug-information parsing, which are the domain of gdb.
- Kernel-level debugging (kernel modules, eBPF programs, kprobes) and tracing of other processes' threads created after launch (multi-threaded syscall tracing).
- Architectures other than x86_64 (the register names and system-call numbers are architecture specific).
- Network analysis and security hardening features.

1.3.3 Assumptions and constraints

- The tool is run on Linux kernel 4.x or newer with the /proc filesystem mounted (standard on all distributions).
- Inspection of processes owned by other users requires sufficient privilege (root or CAP_SYS_PTRACE); tracing and crash analysis always work for programs that LSDT itself launches.
- Process IDs are sampled at the moment of the call; a process may exit between two operations, so every read is checked for errors.

1.4 Objectives of the Project

The project was carried out with the following measurable objectives:

1. To study and apply the Linux kernel interfaces for observation of processes: the /proc filesystem, the ptrace system call, signals, pipes and POSIX threads.
2. To design a modular architecture in which every diagnostic capability is an independent module sharing a small set of helper functions.
3. To implement a **process inspector**, **file-descriptor inspector**, **thread inspector** and **system overview** by parsing /proc files.
4. To implement a **memory-map analyser** that classifies address-space regions and reports RSS, PSS and private-dirty memory.
5. To implement a **system-call tracer** that intercepts every system call of a child process using PTRACE_SYSCALL, decodes the call name, arguments and result, and reports statistics.
6. To implement a **crash analyser** that reports signal, si_code, fault address, registers and the memory region of the faulting instruction.
7. To implement a **live resource monitor** that demonstrates the producer-consumer pattern with a pipe and a mutex.
8. To keep the complete tool below 700 lines of C, free of compiler warnings with `-Wall -Wextra`, and free of dynamic memory allocation so that it cannot leak memory.
9. To validate every module with purpose-built test programs (a NULL-pointer dereference, a divide-by-zero and a multi-threaded CPU/memory consumer) and to measure the overhead of tracing.
10. To document the design, implementation and experimental results in a form that can be used as a teaching aid for Operating Systems laboratories.

1.5 Importance of the Application

The importance of LSDT can be seen from academic, practical and professional viewpoints.

1.5.1 Academic importance

The tool turns abstract Operating System topics into running, observable software. Concepts such as the process control block, virtual memory segments, copy-on-write and demand paging, signals, system-call interface, context switching, inter-process communication and mutual exclusion are all visible in the output of the tool. Because the code is short and free of external dependencies, students can read every line, change it and see the effect, which is far more instructive than using a ready-made binary.

1.5.2 Practical importance

- **Faster diagnosis:** the crash analyser converts an unhelpful "Segmentation fault" into a precise report (for example, "SEGV_MAPERR at address 0x0, instruction in region r-xp of the executable"), which tells the programmer immediately that a NULL pointer was dereferenced.
- **Hidden-failure detection:** the tracer exposes system calls that fail silently. In our experiment, running `cat` on a missing file shows the exact `openat` call that returned ENOENT.

- **Resource-leak detection:** the live monitor shows resident memory growing sample after sample, which is the signature of a memory leak; the descriptor inspector reveals leaked file descriptors.
- **Low overhead and portability:** a 39 KB binary with no dependencies can be copied to a minimal server, a container or a rescue shell where installing gdb or strace is not possible.

1.5.3 Professional importance

Site-reliability engineers, system administrators and security analysts routinely perform exactly the tasks that LSDT automates: finding out what a process is doing, why it is slow, what files it has open and why it terminated. Understanding how such tools work internally makes professionals more effective when they use the industrial-strength equivalents.

1.6 Target Users / Audience

User group	How they use LSDT	Benefit
Engineering students (OS / Systems courses)	Run the tool on sample programs to see processes, memory maps, system calls and signals in action; read the source to learn ptrace and /proc.	Hands-on understanding of Operating System concepts
Faculty and lab instructors	Use the tool and the test programs for demonstrations and lab exercises on process management, memory and signals.	Ready-made teaching aid
Application developers (C / C++ / systems)	Diagnose crashes, unexpected exits, file-open failures and memory growth in their own programs.	Faster bug location
System administrators and DevOps / SRE engineers	Quickly inspect a misbehaving process on a server or inside a container with no extra packages.	Reduced downtime
Security and malware analysts (learners)	Observe the system calls and resources of an unknown program in a controlled environment.	Behaviour profiling
Open-source contributors and hobbyists	Extend the tool with new modules (for example, network sockets or signal tracing).	A clean base to build on

Organisation of the Report

Chapter 2 reviews the Operating System concepts and existing techniques on which the tool is based and describes the proposed approach. Chapter 3 lists the hardware, software and environment requirements. Chapter 4 describes the technologies used. Chapter 5 presents the architecture and design with diagrams. Chapter 6 gives the implementation, including algorithms and the core C code. Chapter 7 shows compilation, execution, test cases and screenshots with real measured output. Chapter 8 concludes the work with results, learning outcomes, limitations and future scope, and Chapter 9 lists the references.

CHAPTER - 2
OS CONCEPTS AND BACKGROUND STUDY

2.1 Operating System Concepts Related to the Project

LSDT is built directly on the services of the operating system. This section explains each concept that the tool uses, so that the design in later chapters can be followed without any outside reference.

2.1.1 User mode, kernel mode and the system-call interface

Modern processors run code in at least two privilege levels. **Kernel mode** can execute any instruction and access all memory; **user mode** is restricted. Application programs, including LSDT, run in user mode. Whenever a program needs a service that only the kernel can provide (reading a file, creating a process, allocating memory), it executes a special instruction that transfers control to the kernel. On x86_64 Linux this instruction is `syscall`. The kernel identifies the requested service from a number placed in register `RAX`, takes up to six arguments from registers `RDI, RSI, RDX, R10, R8, R9`, performs the work and returns the result in `RAX`. A negative value between -4095 and -1 means an error, and its absolute value is the `errno` code. LSDT's tracer relies exactly on this convention to decode system calls.

Register	Role at syscall entry	Role at syscall exit	Field in struct user_regs_struct
RAX	System-call number (also saved as ORIG_RAX)	Return value or -errno	`rax`, `orig_rax`
RDI	Argument 1	-	`rdi`
RSI	Argument 2	-	`rsi`
RDX	Argument 3	-	`rdx`
R10, R8, R9	Arguments 4, 5, 6	-	`r10`, `r8`, `r9`
RIP	Address of next user instruction	-	`rip`

2.1.2 Processes and the process control block

A **process** is a program in execution. For every process the kernel maintains a data structure (the `task_struct` in Linux, commonly called the process control block) that records its identifier (PID), parent (PPID), state, credentials, scheduling information, open files, address space and accounting data. Much of this information is exported to user space through /proc, and LSDT's process inspector simply reads it back. The scheduling states reported by Linux are:

Code	State	Meaning
R	Running / runnable	Executing on a CPU or waiting in the run queue
S	Interruptible sleep	Waiting for an event (I/O, timer, child exit); can be woken by a signal
D	Uninterruptible sleep	Waiting in the kernel, usually for disk I/O; cannot be interrupted
T / t	Stopped / traced	Stopped by a signal or by a debugger (ptrace stop)
Z	Zombie	Has terminated but the parent has not yet collected its exit status with wait()
X	Dead	Being removed; never normally visible

New processes are created with `fork()` (a copy of the parent) and usually followed by `exec()` (replace the program image). The parent collects the exit status with `wait()` or `waitpid()`. LSDT's

tracer and crash analyser follow precisely this fork-exec-wait life cycle, with the addition of ptrace.

2.1.3 Threads

A **thread** is an independent flow of control inside a process. Threads of one process share the same address space, open files and signal handlers but each has its own stack, registers and scheduling state. In Linux, threads are created with the `clone()` system call; the kernel treats each one as a separate task with its own **TID**, while all threads share a common **thread-group ID (TGID)**, which is what user programs call the PID. The kernel exposes every thread under `/proc/PID/task/TID`. LSDT's thread inspector lists these entries, and the live monitor itself uses two POSIX threads.

2.1.4 Virtual memory and the process address space

Each process sees a private, contiguous **virtual address space** that the memory-management unit translates to physical memory page by page. Pages are loaded **on demand**: a page is brought into RAM only when it is first touched, at which point a page fault occurs. The address space is divided into regions, each of which has permissions (read, write, execute) and may be backed by a file or by anonymous memory. Figure 2.1 shows the usual layout, and the right-hand labels show how the memory-map analyser of LSDT names each region.

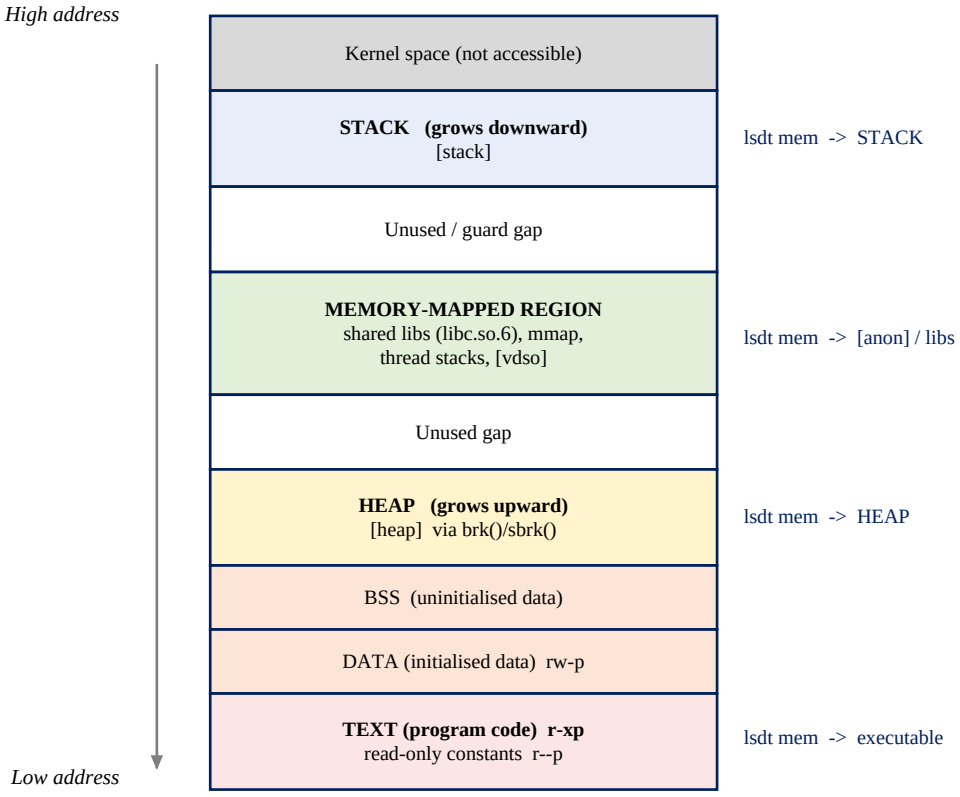


Figure 2.1: Typical virtual address-space layout of a Linux process and the labels used by the LSDT memory analyser

Region	Created by	Typical /proc/PID/maps appearance	Notes
Text (code)	execve loads the ELF file	r-xp, path of the executable	Read-only and shared between processes running the same program

Region	Created by	Typical /proc/PID/maps appearance	Notes
Data / BSS	execve	rw-p, path of the executable or anonymous	Global and static variables
Heap	`brk()` / `sbrk()` used by `malloc`	`[heap]`, rw-p	Grows upward; large allocations use mmap instead
Memory-mapped area	`mmap()`; the dynamic loader maps shared libraries	Paths such as libc.so.6 and anonymous `rw-p` areas	Thread stacks and large `malloc` blocks appear here
Stack	Kernel at exec time; grows on demand	`[stack]`, rw-p	Holds local variables and return addresses
Special	Kernel	`[vdso]`, `[vvar]`	Kernel-provided pages that speed up calls such as clock_gettime
Guard pages	`mprotect` / pthread library	`---p` mappings	Any access raises SIGSEGV; detects stack overflow

Three different "sizes" are commonly confused, and LSDT reports all of them: **VmSize** (total virtual address space, including never-touched pages), **VmRSS / Rss** (resident set size: pages currently in RAM) and **Pss** (proportional set size: resident pages with shared pages divided among the processes sharing them, which is the fairest measure of real memory cost). The Chapter 7 output shows a test process with a virtual size of 230,040 kB but a resident size of only 7,896 kB, a direct illustration of demand paging.

2.1.5 Signals

A **signal** is a software notification delivered to a process. Signals are raised by the kernel (for example, after a CPU exception), by other processes (`kill`), or by the process itself (`raise`, `abort`). If the process has not installed a handler, a default action is applied, which for many signals is termination, optionally with a core dump. The signals most relevant to crash analysis are:

Signal	Number	Typical cause	si_code values reported by LSDT
SIGSEGV	11	Invalid memory reference: NULL pointer, freed memory, stack overflow	SEGV_MAPERR (1): address not mapped; SEGV_ACCERR (2): permission violation
SIGFPE	8	Arithmetic error: integer divide by zero, overflow	FPE_INTDIV (1): integer divide by zero
SIGABRT	6	`abort()`, failed `assert()`, glibc heap-corruption detection	SI_TKILL / SI_USER
SIGILL	4	Execution of an illegal or privileged instruction	ILL_ILLOPC and others
SIGBUS	7	Misaligned or non-existent physical memory access, truncated mmap file	BUS_ADRALN, BUS_ADRERR

Signals produced directly by a CPU exception are called **synchronous** and are delivered to the faulting thread together with a `siginfo_t` structure that contains the faulting address (`si_addr`). When a process is being traced, the tracer is notified before the process sees the signal, which allows LSDT to inspect the registers at the exact moment of the fault.

2.1.6 File descriptors and the virtual file system

In UNIX "everything is a file". A **file descriptor** is a small non-negative integer that indexes the process's table of open files; descriptors 0, 1 and 2 are standard input, output and error. The same table holds regular files, directories, pipes, sockets and devices. The kernel exposes the table as symbolic links in `/proc/PID/fd`, which LSDT reads with `readlink()`. A process that opens files without closing them eventually reaches its `RLIMIT_NOFILE` limit and fails with EMFILE, a classic resource-leak bug that the descriptor inspector can reveal.

2.1.7 The /proc pseudo-filesystem

`/proc` is a virtual filesystem generated on the fly by the kernel; no data is stored on disk. Reading a file in /proc causes the kernel to format its current internal state as text. The idea of exposing processes as files was introduced in the UNIX research literature in 1984 (Killian) and is now a core Linux interface. Table 2.1 lists the /proc files read by LSDT.

File	Content	Used by LSDT module
/proc/PID/status	Human-readable process state: Name, State, PPid, Uid, Threads, Vm*, context switches	Process inspector
/proc/PID/stat	Machine-readable single line: state, utime, stime, num_threads, rss ...	Live monitor, thread inspector
/proc/PID/cmdline, exe, cwd	Arguments (NUL separated), executable and working directory (symbolic links)	Process inspector
/proc/PID/limits	Resource limits such as Max open files	Process inspector
/proc/PID/fd/	One symbolic link per open file descriptor	File-descriptor inspector
/proc/PID/task/TID/stat	Per-thread statistics	Thread inspector
/proc/PID/maps	Every memory mapping: range, permissions, offset, device, inode, path	Memory analyser, crash analyser
/proc/PID/smaps_rollup	Summed Rss, Pss, Shared_Clean, Private_Dirty, Swap	Memory analyser
/proc/loadavg, uptime, meminfo	System-wide load, uptime and memory statistics	System overview

Table 2.1: /proc files used by the tool

2.1.8 Process tracing with ptrace

`ptrace(2)` is the system call on which all UNIX debuggers are built. A **tracee** is placed under the control of a **tracer**. The kernel then stops the tracee at interesting events and notifies the tracer through `waitpid()`. The tracer may read and modify the tracee's memory and registers and then resume it. Two resumption requests are important for LSDT: `PTRACE_SYSCALL` resumes the tracee until the next system-call entry or exit, and `PTRACE_CONT` resumes it until the next signal. Because the tracer is notified twice for every call (before and after the kernel executes it), a simple toggle variable distinguishes entry from exit. Setting the option `PTRACE_O_TRACESYSGOOD` marks such stops as `SIGTRAP | 0x80`, so they cannot be confused with an ordinary SIGTRAP.

2.1.9 CPU scheduling and accounting

The scheduler divides processor time among runnable tasks. The kernel counts CPU time consumed by each task in clock ticks (jiffies), of which there are `sysconf(_SC_CLK_TCK)` per second (normally 100). The fields `utime` (time in user mode) and `stime` (time in kernel mode) in /proc/PID/stat are cumulative. A tool computes CPU utilisation by sampling the counter twice:

```
CPU% = 100 x ( (utime2 + stime2) - (utime1 + stime1) ) / CLK_TCK  /  (t2 - t1)
```

The status file additionally reports **voluntary** context switches (the process blocked or yielded) and **non-voluntary** context switches (the scheduler pre-empted it), which help to distinguish I/O-bound from CPU-bound behaviour.

2.1.10 Inter-process communication and synchronisation

A **pipe** is a kernel buffer with a read end and a write end; data written at one end can be read at the other, in order. A **mutex** is a lock that ensures only one thread at a time executes a critical section, preventing race conditions on shared data. Together they implement the classic **producer-consumer problem** that appears in every Operating Systems text-book. LSDT's live monitor uses a sampler thread (producer), a logger thread (consumer), a pipe as the bounded buffer and a mutex to protect the shared statistics structure (Section 6.6).

2.2 Existing Methods / Algorithms

Linux already offers many debugging and monitoring tools. They are summarised below because LSDT is both inspired by them and positioned against them.

Tool	Purpose	Underlying mechanism	Limitations for learning / small systems
`strace`	Trace system calls and signals	ptrace (PTRACE_SYSCALL / seccomp)	Large code base; only tracing; separate tool for everything else
`ltrace`	Trace library calls	ptrace + breakpoints on PLT entries	Architecture and library dependent
`gdb`	Source-level debugging, breakpoints, core files	ptrace + DWARF debug information	Steep learning curve; needs debug symbols
`valgrind`	Memory error and leak detection	Dynamic binary instrumentation (JIT translation)	10-50x slowdown; heavy; not for production
`AddressSanitizer`	Detect memory errors at run time	Compiler instrumentation + shadow memory	Requires recompilation
`ps`, `top`, `htop`	Process list and live resource view	/proc parsing	No tracing or crash analysis
`lsof`	Open files and sockets	/proc/PID/fd and kernel tables	Output is verbose; no memory or CPU data
`pmap`	Process memory map	/proc/PID/maps, smaps	Memory only
`perf`, `eBPF/bpftrace`, `LTTng`	Profiling and kernel-wide tracing	Hardware counters, kernel probes, ring buffers	Require privileges and kernel support; complex
Core dump + `gdb`	Post-mortem analysis	Kernel writes memory image on fatal signal	Disabled by default (`ulimit -c 0`); large files

2.2.1 Algorithm used by system-call tracers

Tracers such as strace use the following algorithm, which LSDT re-implements in a compact form: (1) fork a child; (2) in the child call `ptrace(PTRACE_TRACEME)` and `exec` the target; (3) in the parent wait for the child to stop; (4) repeat `ptrace(PTRACE_SYSCALL)` followed by `waitpid()`; (5) at each stop read the registers with `PTRACE_GETREGS`; (6) alternately treat the stop as entry (read the call number and arguments) and exit (read the return value); (7) stop when `waitpid` reports that the child has exited.

2.2.2 Sampling algorithm used by process monitors

Tools such as `top` periodically read /proc/PID/stat, subtract the previous cumulative CPU counters and divide by the elapsed wall-clock time. The accuracy depends on the sampling interval; very short intervals are noisy because the kernel counts time in whole ticks, while very long intervals hide short bursts. LSDT uses a default interval of 500 ms and allows the user to change it.

2.2.3 Post-mortem analysis

Traditional crash analysis relies on a **core dump**: when a process dies from a fatal signal the kernel writes its memory image to a file, which is later examined with gdb. This requires the core-file size limit to be enabled and the debugger to be available, and it yields a very large file. An alternative, used by LSDT, is to supervise the process with ptrace so that the tracer is informed of the fatal signal **before** the process dies, and to read the registers and `siginfo_t` directly, producing a short text report immediately.

2.2.4 Limitations of the existing approaches

- Each tool covers one aspect; correlating tracing, memory and resource data requires several tools and manual effort.
- Their source code is too large to be used as a learning resource.
- Output formats are different, making scripting and comparison harder.
- Several need extra packages, special privileges or recompilation, which is impractical on minimal servers and containers.

2.3 Proposed Approach

LSDT proposes a **unified, modular, dependency-free user-space debugger** that implements the essential algorithms of the tools above in one small C program. The approach is based on five design principles:

1. **Use the kernel as the single source of truth.** All data comes from /proc, ptrace or signals; nothing is guessed or estimated by the tool.
2. **One binary, many sub-commands.** The user learns one command name (`lsdt`) and one consistent output style.
3. **Modular design.** Each capability is a self-contained function (`cmd_info`, `cmd_trace`, `cmd_crash`, ...) built on shared helpers.
4. **Safe, allocation-free code.** Fixed-size buffers, bounded `snprintf`, checked system-call returns and no `malloc` means that the tool itself cannot leak memory or overflow the heap.

5. **Readable by students.** Short functions, plain C and comments allow every line to be understood and modified.

Functionally the proposed system works as follows. For **static inspection** (`info`, `fds`, `threads`, `mem`, `sys`) the tool opens the relevant /proc files, parses the text and prints an aligned report. For **dynamic analysis** (`trace`, `crash`) it launches the target program as its own child under ptrace, intercepts system calls or signals, reads CPU registers and target memory and prints the findings. For **monitoring** (`watch`) it starts a sampler thread that reads /proc/PID/stat every interval and sends one record per sample through a pipe to a logger thread that prints and logs it, using a mutex for shared counters. The detailed design is in Chapter 5 and the implementation in Chapter 6.

2.4 Advantages of the Proposed Approach

- **Integration:** eight diagnostic functions behind one command, with consistent output.
- **Transparency:** the whole implementation (about 600 lines) can be read in an afternoon; every OS concept is visible in the code.
- **Light weight:** the compiled binary is about 39 KB and starts instantly; there is no runtime, interpreter or library to install.
- **Low risk:** the tool only reads kernel information or traces its own child process; it never modifies other processes, so it is safe to use on a live system.
- **Immediate crash reports:** no core-dump configuration or symbol files are needed; the fault address, instruction pointer and memory region are printed at the moment of the crash.
- **Educational value:** each module maps directly to a topic of the Operating Systems syllabus (Section 7.5).
- **Extensible:** a new module needs one function and one line in the dispatcher.

Feature comparison

Feature	ps/top	lsof	pmap	strace	gdb	LSDT
Process details	Yes	No	No	No	Partial	Yes
Open file descriptors	No	Yes	No	No	Partial	Yes
Thread listing	Yes	No	No	No	Yes	Yes
Memory-map classification	No	No	Yes	No	Yes	Yes
System-call trace + statistics	No	No	No	Yes	Partial	Yes
Fault address / registers on crash	No	No	No	No	Yes	Yes
Live CPU / RSS log to file	Yes	No	No	No	No	Yes
No dependencies, tiny source	No	No	No	No	No	Yes
Source-level debugging, breakpoints	No	No	No	No	Yes	No

LSDT does not replace gdb or strace; it deliberately covers the most common diagnostic needs in a form that is small enough to be understood completely.

CHAPTER - 3
SYSTEM REQUIREMENTS

3.1 Hardware Requirements

LSDT is a very light program; its hardware requirements are those of a modern laptop running Ubuntu. The tool itself uses a few hundred kilobytes of memory and negligible CPU time. The table gives the minimum and recommended configurations for development, and the configuration on which the results of this report were measured.

Component	Minimum	Recommended	Test machine used for results
Processor	x86_64 (64-bit Intel / AMD), 1 core	Dual-core or better, 2 GHz+	x86_64 virtual machine, 1 vCPU
Main memory (RAM)	2 GB	4 GB or more (8 GB if WSL 2 and an IDE are used together)	4 GB (4,093,768 kB reported by /proc/meminfo)
Disk space	1 GB free (OS tools, compiler)	5 GB free (Ubuntu + WSL + editor)	Less than 1 MB used by the tool and tests
Display / input	Any terminal	Full-HD display, keyboard and mouse	Terminal only
Network	Not needed to run LSDT	Internet for installing packages	Used only for installing the compiler
Architecture note	The system-call numbers and register names in the source are for **x86_64**	-	x86_64

3.2 Software Requirements

Software	Version used	Purpose	Mandatory?
Operating system	Ubuntu 24.04 LTS (Linux kernel 6.x)	Execution environment that provides /proc, ptrace and signals	Yes
GNU C Compiler (gcc)	13.3.0	Compile `lsdt.c` and the test programs	Yes
GNU C Library (glibc) + pthreads	2.39	System-call wrappers, POSIX threads (`-pthread`)	Yes (bundled)
GNU Make	4.3	Automate the build with the Makefile	Recommended
GNU Debugger (gdb)	Ubuntu repository version	Optional: compare crash addresses and learn source-level debugging	Optional
strace	Ubuntu repository version	Optional: compare its output with the LSDT tracer	Optional
binutils (objdump, readelf, addr2line)	2.42	Disassemble the faulting instruction and inspect the ELF file	Optional
Git	2.43	Version control of the source code	Recommended
Text editor / IDE	VS Code (with WSL extension) / vim / nano	Edit the source code	Any one
Python 3 with matplotlib	3.12 / 3.10	Draw the charts in this report from the monitor log (not needed to run LSDT)	Optional

3.3 Operating System Environment - Ubuntu / WSL

LSDT depends on Linux-specific interfaces and therefore must run on Linux. Two equally supported ways of obtaining a suitable environment are described below.

3.3.1 Native Ubuntu

Ubuntu is a Debian-based distribution with long-term support. Install it on a physical machine, dual-boot or a virtual machine (VirtualBox, VMware, KVM). The default kernel includes the /proc filesystem, ptrace support and all signal features that the tool needs.

3.3.2 Ubuntu on Windows Subsystem for Linux (WSL 2)

Students working on Windows laptops can use WSL 2, which runs a genuine Linux kernel inside a lightweight virtual machine. Because the kernel is real, /proc, ptrace and signals behave exactly as on native Ubuntu. (WSL 1 translates Linux system calls to Windows and has gaps in its /proc and ptrace behaviour, so WSL 2 is recommended.) Installation takes three steps:

```
PS> wsl --install -d Ubuntu-24.04        # run in Windows PowerShell (administrator)
PS> wsl --set-version Ubuntu-24.04 2   # make sure WSL 2 is used
$   sudo apt update && sudo apt install -y build-essential gdb strace make git
```

3.3.3 Permissions and security settings that affect the tool

- **Tracing your own children** (the `trace` and `crash` commands) is always allowed, regardless of the Yama security module, because LSDT forks the target itself.
- **Reading /proc of other processes:** most files of processes owned by the same user are readable; `fd`, `maps` and `smaps_rollup` of other users' processes require root or CAP_SYS_PTRACE (use `sudo`).
- **Yama ptrace_scope** (`/proc/sys/kernel/yama/ptrace_scope`): value 1 on Ubuntu restricts attaching to unrelated processes; this does not affect LSDT because it never attaches to an existing process.
- **Containers:** depending on the Docker and seccomp version, ptrace may be blocked inside a container; start it with `--cap-add=SYS_PTRACE` if tracing is required.
- **Core dumps are not needed.** Unlike gdb-based post-mortem analysis, the crash analyser does not require `ulimit -c unlimited`.

3.3.4 Verifying the environment

```
$ uname -srm            # kernel and architecture (must show x86_64)
$ lsb_release -d        # Ubuntu version
$ gcc --version         # compiler
$ ls /proc/self/status  # /proc is available
$ nproc; free -m        # CPUs and memory
```

3.4 Development Tools and Compilers

The tool chain used during development is the standard GNU tool chain. The main compiler options used for building LSDT are explained below; each one has a specific purpose in producing a reliable program.

Option	Meaning	Why it is used in this project
`-Wall -Wextra`	Enable the common and extra warning groups	Catches unused variables, sign problems and unsafe format strings. The build is warning-free.

Option	Meaning	Why it is used in this project
`-O2`	Optimisation level 2	Produces fast code for the sampling loops without changing behaviour
`-pthread`	Define thread-safe macros and link libpthread	Needed by the live monitor (`pthread_create`, `pthread_mutex_lock`)
`-g -O0`	Debug information, no optimisation	Used for the test programs so that crash locations map to source lines in gdb
`-o lsdt`	Name of the output executable	-
`_GNU_SOURCE` **(macro in source)**	Enable GNU/Linux extensions	Makes GNU extensions such as `strerrorname_np()` and `usleep()` visible

Supporting tools were used as follows: **make** to rebuild only what changed; **objdump -d** to disassemble the test programs and confirm that the instruction pointer reported by the crash analyser is exactly the faulting machine instruction (Section 7.5: an `idivl` for the divide-by-zero test and a `movl` store through a NULL pointer for the segmentation-fault test); **readelf** to confirm that the test programs are position-independent executables (which explains why the reported addresses change on every run); and **git** for version control among the three team members. **gdb** and **strace** can be used in the same way to compare the output of LSDT with that of the professional tools.

Build files

```
CC=gcc
CFLAGS=-Wall -Wextra -O2 -pthread
all: lsdt tests
lsdt: lsdt.c
▫$(CC) $(CFLAGS) -o lsdt lsdt.c
tests: crash_segv crash_div busy_threads
crash_segv: tests/crash_segv.c
▫$(CC) -g -O0 -o $@ $<
crash_div: tests/crash_div.c
▫$(CC) -g -O0 -o $@ $<
busy_threads: tests/busy_threads.c
▫$(CC) -O1 -pthread -o $@ $<
clean:
▫rm -f lsdt crash_segv crash_div busy_threads lsdt_monitor.log
```

CHAPTER - 4
TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The entire tool is written in the **C programming language** (GNU C11 dialect). C was chosen deliberately, and not only because the Linux kernel and its system-call interface are themselves defined in C terms. The reasons are:

- **Direct access to the kernel interface.** System calls such as `ptrace`, `waitpid` and `pipe` are declared in C headers, and structures such as `struct user_regs_struct` and `siginfo_t` have a C memory layout that the kernel fills in directly.
- **Predictable resource use.** C has no garbage collector or runtime; the programmer controls every byte, which is essential when a debugging tool must not disturb the program it observes.
- **Small, dependency-free binary.** The compiled tool is about 39 KB and needs only the C library that is present on every Linux system.
- **Educational transparency.** Every system-call invocation is visible in the source, so a reader can see exactly which operating-system service produces each line of output.

C++ was considered because it offers classes, the standard library containers and RAII for resource management. It was not adopted for the final tool because the project does not need object-oriented abstractions, the allocation-free design removes the need for RAII, and keeping the code in plain C makes the mapping to the kernel interface obvious. The test programs and the tool can be compiled unchanged as C++ in most places, and a C++ wrapper is mentioned as future work.

4.1.1 C language features used

Feature	Where it is used in LSDT
Structures (`struct`)	`struct sysstat` (per-syscall statistics), `struct monitor` (shared state of the live monitor), `struct user_regs_struct`, `siginfo_t`
Pointers and pointer arithmetic	Parsing /proc text (`strchr`, `strrchr`), walking `argv`, `status_field()` returns text copied from the middle of a buffer
Fixed-size arrays and `sizeof`	Every buffer is sized at compile time (`char buf[8192]`), and `sizeof` is passed to `snprintf` and `read` for bounds checking
Function pointers / thread entry points	`sampler_thread` and `logger_thread` have the `void *(*)(void *)` signature required by `pthread_create`
Switch statements on integers	`sysname()` and `signame()` convert numbers to readable names
Designated initialisers	`struct monitor m = { .pid = pid, .interval_ms = ... }`
Bit operations	`SIGTRAP | 0x80` test and extraction of bytes from the word returned by `PTRACE_PEEKDATA`
Preprocessor macros	`_GNU_SOURCE`, `MAXSYS`, status macros `WIFEXITED`, `WSTOPSIG` ...

4.2 GCC Compiler

The **GNU Compiler Collection (GCC)**, version 13.3.0 on Ubuntu 24.04, translates the C source into an executable. Compilation consists of four stages, each of which can be observed with the corresponding flag:

Stage	Tool	Flag to stop here	Result
1. Pre-processing	cpp	`gcc -E lsdt.c`	Headers included, macros expanded
2. Compilation	cc1	`gcc -S lsdt.c`	x86_64 assembly (`lsdt.s`)
3. Assembly	as	`gcc -c lsdt.c`	Relocatable object file (`lsdt.o`)
4. Linking	ld	`gcc lsdt.o -pthread`	Executable ELF file linked with libc and libpthread

GCC's diagnostics were useful during development. For example, the first build reported `-Wformat-truncation` warnings in the descriptor and thread inspectors: a 128-byte path buffer could theoretically be too small for a long directory-entry name. The buffers were enlarged to 512 bytes, and the program now compiles with **zero warnings** under `-Wall -Wextra`. The test programs are compiled with `-g -O0` so that the compiler emits debug information and does not reorder instructions, which keeps the faulting instruction predictable for the crash-analysis experiments.

4.3 Linux System Calls

Every service that LSDT obtains from the kernel is a **system call**, usually invoked through a thin wrapper function in the C library. The following table lists all system calls and wrappers used by the tool and the module in which each is used.

System call / wrapper	Purpose	Used in
`open`, `read`, `close`	Read /proc text files into a buffer	`read_file()` (all /proc readers)
`readlink`	Resolve symbolic links /proc/PID/exe, cwd and fd/N	info, fds
`opendir`, `readdir` **(getdents64)**	Enumerate /proc/PID/fd, /proc/PID/task and /proc	fds, threads, sys
`stat`	Find the file type behind a descriptor	fds
`fork`	Create the child that will be traced	trace, crash
`ptrace`	Trace and control the child (details below)	trace, crash
`execvp` **(execve)**	Replace the child image with the target program	trace, crash
`raise`	Make the child stop itself with SIGSTOP	`spawn_traced()`
`waitpid` **(wait4)**	Wait for the child to stop, exit or be killed; obtain its status	trace, crash
`kill`	Terminate the crashed child with SIGKILL	crash
`pipe`	Create the pipe between sampler and logger	watch
`write`	Send a sample record into the pipe	sampler thread
`pthread_create` **(clone)**, `pthread_join`	Create and wait for the two monitor threads	watch
`pthread_mutex_lock / unlock`	Mutual exclusion on shared counters (futex)	watch
`clock_gettime`	High-resolution time for syscall duration and sampling interval	trace, watch
`usleep` **(nanosleep)**	Sleep between samples	watch
`sysconf`	Clock ticks per second (`_SC_CLK_TCK`) and page size	watch

4.3.1 The ptrace requests used

Request	Meaning	Used by
`PTRACE_TRACEME`	Called by the child: "let my parent trace me"	spawn_traced()
`PTRACE_SETOPTIONS` **with** `PTRACE_O_TRACESYSGOOD`	Mark syscall stops as `SIGTRAP | 0x80`	spawn_traced()
`PTRACE_SYSCALL`	Resume the tracee until the next syscall entry or exit	cmd_trace()
`PTRACE_CONT`	Resume the tracee until the next signal; optionally deliver a signal	cmd_crash()
`PTRACE_GETREGS`	Copy the CPU registers into `struct user_regs_struct`	trace, crash
`PTRACE_GETSIGINFO`	Retrieve the `siginfo_t` of the signal that stopped the tracee (fault address, si_code)	cmd_crash()
`PTRACE_PEEKDATA`	Read one machine word from the tracee's memory (used to read file-name arguments)	peek_string()

4.3.2 Interpreting the wait status

`waitpid()` returns a packed integer. The C library supplies macros to decode it, and LSDT uses all of the following:

Macro	True when	Handled as
`WIFEXITED(st)` / `WEXITSTATUS(st)`	Child called exit() or returned from main	Print "exited with N"; stop the loop
`WIFSIGNALED(st)` / `WTERMSIG(st)`	Child was killed by a signal that the tracer did not intercept	Print "killed by signal N"
`WIFSTOPPED(st)` / `WSTOPSIG(st)`	Child is stopped under ptrace	Inspect the signal number
`WSTOPSIG(st) == (SIGTRAP | 0x80)`	Syscall entry or exit stop (TRACESYSGOOD)	Read registers and toggle entry / exit

4.4 POSIX Threads / Synchronization

POSIX threads (pthreads) are the standard C interface for multi-threaded programming on Linux. The live monitor (`watch`) uses three pthread facilities:

- `pthread_create()` / `pthread_join()` to start the sampler and logger threads and wait for them to finish. The thread entry functions take and return a `void *`, and the shared `struct monitor` is passed as the argument.
- `pthread_mutex_t` **with** `pthread_mutex_lock()` / `pthread_mutex_unlock()` to guarantee that the counters `produced`, `consumed`, `peak_cpu` and `peak_rss_kb` are updated by one thread at a time. Without the mutex, the increment `produced++` (a read, an add and a write) could be interleaved between the two threads and an update could be lost.
- **A pipe as the communication buffer.** Instead of a shared queue protected by condition variables, the program uses the kernel's pipe. A pipe already provides blocking, ordering and a bounded buffer, so the consumer automatically sleeps when no data is available and the producer automatically blocks if the buffer is full.

The design follows the classical **producer-consumer** pattern. Termination is signalled by closing the write end of the pipe: when the sampler finishes, the logger's `fgets()` returns `NULL` (end of file) and

the logger exits. The program is compiled with `-pthread`, which defines the reentrant macros and links the thread library.

Synchronisation tool	Alternative considered	Reason for the final choice
Mutex for counters	Atomic variables (`stdatomic.h`)	Mutexes are the technique taught in the course; four related values must be updated together, which atomics cannot do
Pipe for records	Shared ring buffer + condition variable + semaphore	The kernel provides blocking and ordering; far less code and fewer chances of deadlock
EOF to terminate	Special "poison pill" record	Natural pipe behaviour; nothing extra to send or parse

4.5 Linux Commands and Utilities

A number of standard commands were used during development, testing and report preparation. They also serve as the reference against which the output of LSDT can be compared.

Command	Purpose in the project	Example
`cat /proc/PID/status`	Compare with `lsdt info`	`cat /proc/$$/status`
`ls -l /proc/PID/fd`	Compare with `lsdt fds`	`ls -l /proc/$$/fd`
`ps -L -p PID`	List threads; compare with `lsdt threads`	`ps -L -p 351`
`cat /proc/PID/maps`, `pmap PID`	Compare with `lsdt mem`	`pmap -x 351`
`top`, `htop`	Compare CPU % and RES with `lsdt watch`	`top -H -p 351`
`free -m`, `cat /proc/meminfo`	Compare with `lsdt sys`	`free -m`
`kill -SEGV PID`, `kill -l`	Send test signals; list signal numbers	`kill -l 11`
`ulimit -n`, `ulimit -c`	Show resource limits (open files, core size)	`ulimit -n`
`objdump -d`, `readelf -h`	Disassemble test programs; check ELF type	`objdump -d crash_div`
`make`, `gcc`	Build	`make`
`time`	Measure the overhead of tracing	`time ./lsdt trace -c ls`
`man 2 ptrace`, `man 5 proc`, `man 7 signal`	Primary documentation	`man 5 proc`

4.6 Other Tools / Libraries Used

Tool / library	Role
GNU C Library (glibc 2.39)	Supplies the wrappers for all system calls, `snprintf`, `strerror`, `strerrorname_np` and the POSIX thread implementation (NPTL). It is the only library linked.
GNU Make	The `Makefile` builds the tool and the three test programs with one command and provides a `make clean` target.
Git	Version control and collaboration among the three team members.
Visual Studio Code (WSL extension) / vim	Source editing from Windows with the compiler running inside Ubuntu.
Python 3, matplotlib, Pillow, ReportLab	Used only for preparing this report: the charts in Chapter 7 were drawn from the captured output of LSDT, the terminal-style figures were rendered from the real text output, and the report itself was typeset with ReportLab. None of them is needed to build or run LSDT.
Test programs written for the project	`crash_segv.c`, `crash_div.c` and `busy_threads.c` generate the situations that the tool is designed to diagnose.

CHAPTER - 5

SYSTEM ARCHITECTURE AND DESIGN

5.1 Overall System Architecture

LSDT has a **layered, modular architecture** (Figure 5.1). The user interacts only with the top layer, the command-line dispatcher. Below it, eight independent modules perform the diagnostic work. All modules share a small set of helper functions and communicate with the kernel through four families of interfaces: the /proc filesystem, ptrace/waitpid, signals, and pipes/threads.

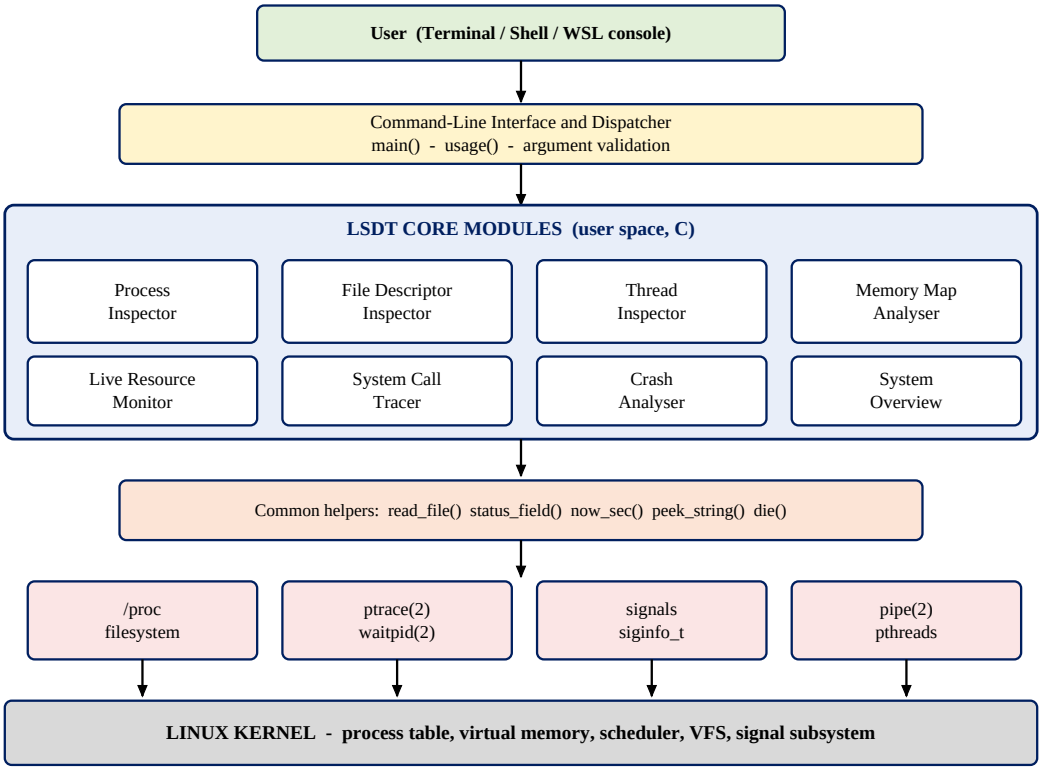


Figure 5.1: Layered architecture of the Linux System Debugging Tool

5.1.1 Layers and responsibilities

Layer	Components	Responsibility
1. User interface layer	Terminal, `main()`, `usage()`	Parse the command line, validate arguments, select the module and return an exit code
2. Module layer	`cmd_info`, `cmd_fds`, `cmd_threads`, `cmd_mem`, `cmd_watch`, `cmd_trace`, `cmd_crash`, `cmd_sys`	Implement the diagnostic functions; each is independent of the others
3. Helper layer	`read_file`, `status_field`, `now_sec`, `peek_string`, `spawn_traced`, `sysname`, `signame`, `describe_addr`	Reusable code for reading /proc, finding keys, timing, reading tracee memory and launching a traced child
4. Kernel interface layer	/proc, ptrace + waitpid, signals + siginfo, pipe + pthreads	The only way in which the tool obtains information; no private kernel data is used
5. Operating system kernel	Process table, virtual memory manager, scheduler, VFS, signal subsystem	Provides and enforces everything above

5.1.2 Design principles

- **Separation of concerns:** data collection (reading the kernel), analysis (classifying, computing) and presentation (printing) are performed in distinct steps inside each module.
- **Loose coupling:** modules never call each other; they use only the helper layer. A module can be removed or replaced without affecting the rest.
- **Fail safe:** every system call return value is checked; a process that vanishes during inspection produces an error message instead of a crash.
- **Bounded memory:** all buffers are fixed-size and all string formatting uses `snprintf` with `sizeof`; there is no dynamic memory allocation.
- **Observer effect minimised:** inspection modules only read; tracing modules touch only the child process that the tool itself created.

5.1.3 Source-code organisation

File	Lines (approx.)	Content
`lsdt.c`	607	The complete tool, organised in eight labelled sections (module 1 to module 8) plus helpers and `main()`
`tests/crash_segv.c`	10	Dereferences a NULL pointer: raises SIGSEGV
`tests/crash_div.c`	12	Integer division by zero: raises SIGFPE
`tests/busy_threads.c`	30	Three threads that consume CPU and grow memory, with one open file
`Makefile`	14	Build rules
`lsdt_monitor.log`	generated	CSV log written by the `watch` command

5.1.4 Principal data structures

```
struct sysstat {            /* one entry per system-call number (index = syscall nr) */
    long   calls;           /* how many times it was invoked                          */
    long   errors;          /* how many returned -errno                               */
    double secs;            /* total time spent inside the kernel for this call       */
};

struct monitor {            /* shared between sampler and logger threads              */
    pid_t pid;              /* process being monitored                                */
    int   interval_ms, samples, wfd, rfd;   /* settings and pipe ends                 */
    pthread_mutex_t lock;   /* protects the four fields below                         */
    long   produced, consumed;
    double peak_cpu;
    long   peak_rss_kb;
};
```

5.2 System Workflow

The overall flow of control of the program is shown in Figure 5.2. Execution always begins in `main()`, which reads `argv[1]` as the command name and checks that the required arguments are present. If the command is unknown or an argument is missing, the usage text is printed and the program ends with exit code 1. Otherwise control passes to exactly one module, which prints its report to standard output and returns a status code.

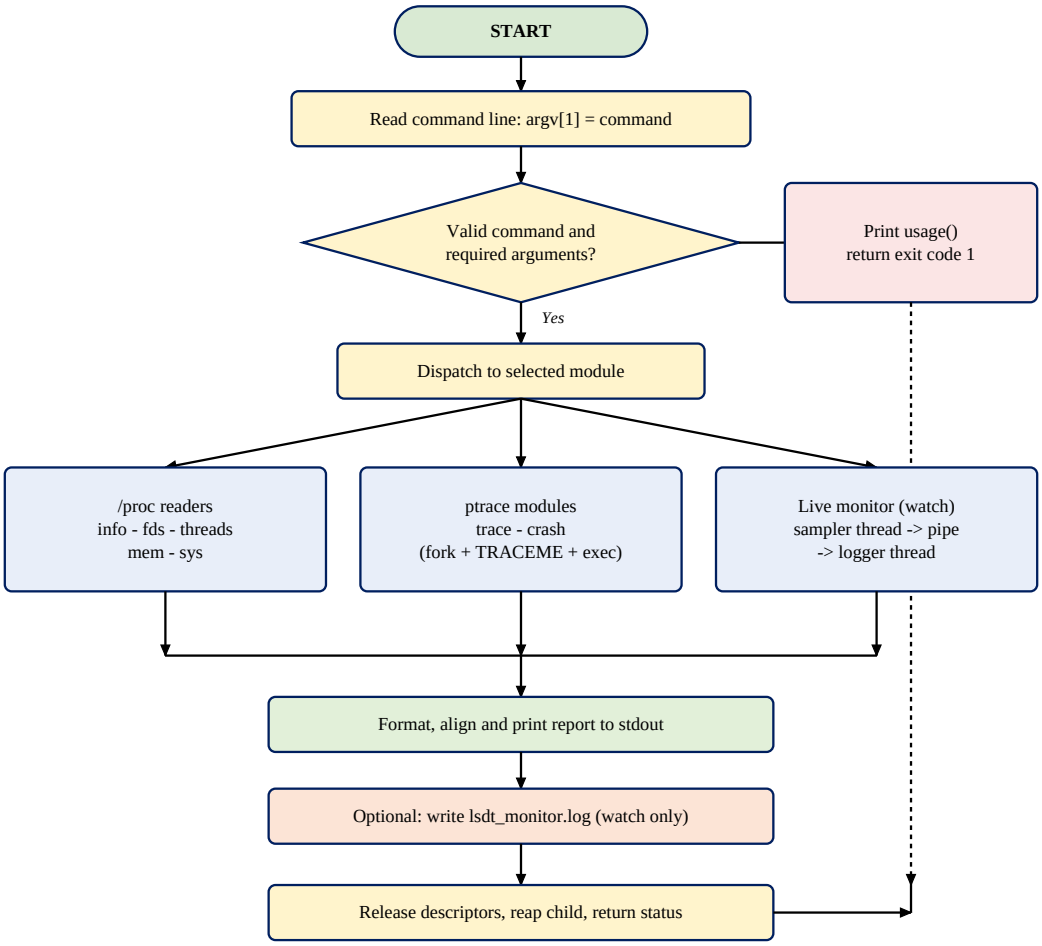


Figure 5.2: Overall workflow of LSDT

5.2.1 Workflow of the /proc-based commands (info, fds, threads, mem, sys)

1. Convert the PID argument with `atoi()`.
2. Build the path of the required /proc file with `snprintf`.
3. Open and read the file (`read_file`), or enumerate the directory (`opendir`/`readdir`).
4. Locate the wanted fields (`status_field`, `sscanf`, `strchr`).
5. Classify or convert the values (for example, map a mapping to heap / stack / file-backed).
6. Print an aligned report and return 0, or print `perror` and return 1 if the process does not exist.

5.2.2 Workflow of the tracing commands (trace, crash)

1. `spawn_traced()` forks a child. The child calls `PTRACE_TRACEME`, stops itself with `raise(SIGSTOP)` and then `execvp()` the target program.
2. The parent waits for the child's first stop and sets `PTRACE_O_TRACESYSGOOD`.
3. In `trace`, the parent loops on `PTRACE_SYSCALL` + `waitpid`; each syscall stop is decoded as entry or exit. In `crash`, the parent loops on `PTRACE_CONT` + `waitpid` and examines each signal stop.
4. On child exit, `trace` prints its statistics table, and `crash` reports "no crash detected". If a fatal signal occurs, `crash` gathers the registers and `siginfo`, prints the report and kills the child.

5.2.3 Workflow of the live monitor (watch)

1. Create a pipe and initialise `struct monitor`.
2. Start the logger thread (consumer), then the sampler thread (producer).
3. The sampler sleeps for the interval, reads /proc/PID/stat, computes CPU % and RSS, writes a CSV record to the pipe and updates the shared peaks under the mutex.
4. The logger reads records from the pipe, prints them as table rows, appends them to `lsdt_monitor.log` and counts them under the mutex.
5. After the requested number of samples the sampler closes the pipe; the logger sees end-of-file; `main` joins both threads and prints the summary.

5.3 Flowchart / Data Flow

Figure 5.3 shows the data-flow diagram of the system. The USER supplies a command (process 1), which is validated and turned into a request. Process 2 collects raw data from two kernel sources: the /proc files (data store D1) and the stop events and registers delivered by ptrace (D2), which are generated by the TARGET program. Process 3 analyses and formats the data and returns a report to the user; in monitoring mode, process 4 additionally writes sample records to the log file (D3).

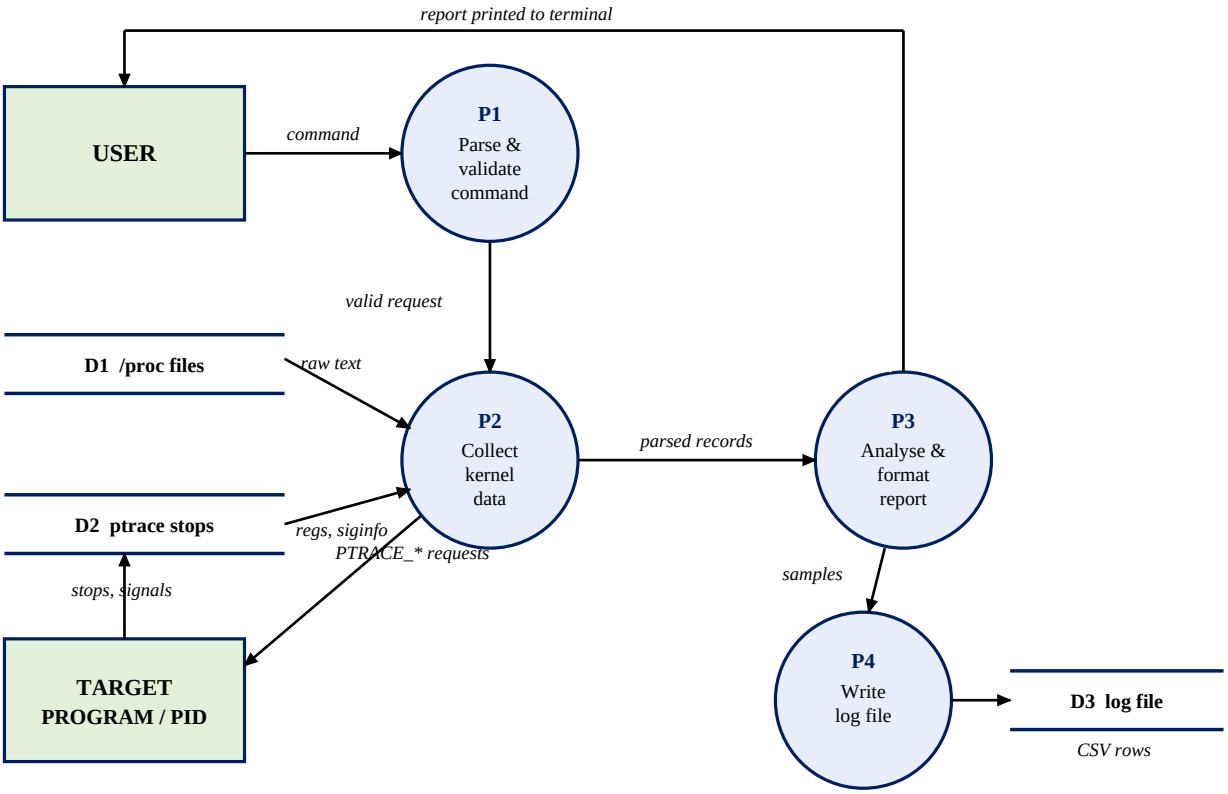


Figure 5.3: Data-flow diagram of LSDT

5.3.1 Data dictionary

Data item	Description	Format / example
command	Sub-command and arguments typed by the user	`trace -c /bin/ls demo_dir`
valid request	Parsed command, PID or program argv	PID integer or `char **argv`
raw text	Unparsed content of a /proc file	`VmRSS: 7896 kB`
regs, siginfo	Registers and signal information from ptrace	`struct user_regs_struct`, `siginfo_t`
parsed records	Numeric fields extracted from raw data	Page counts, jiffies, addresses, syscall numbers
report	Formatted text for the terminal	Aligned columns with a title line starting `===`
CSV rows	One line per monitoring sample	`sample,cpu%,rss_kb,threads` e.g. `1,92.0,10880,4`

5.3.2 Flowchart of the crash analyser

The crash analyser is the most intricate module, so its control flow is shown separately in Figure 5.4. The loop forwards all non-fatal signals (for example SIGCHLD or SIGALRM) to the program unchanged so that its normal behaviour is not altered, and acts only on the five signals that indicate a fatal fault.

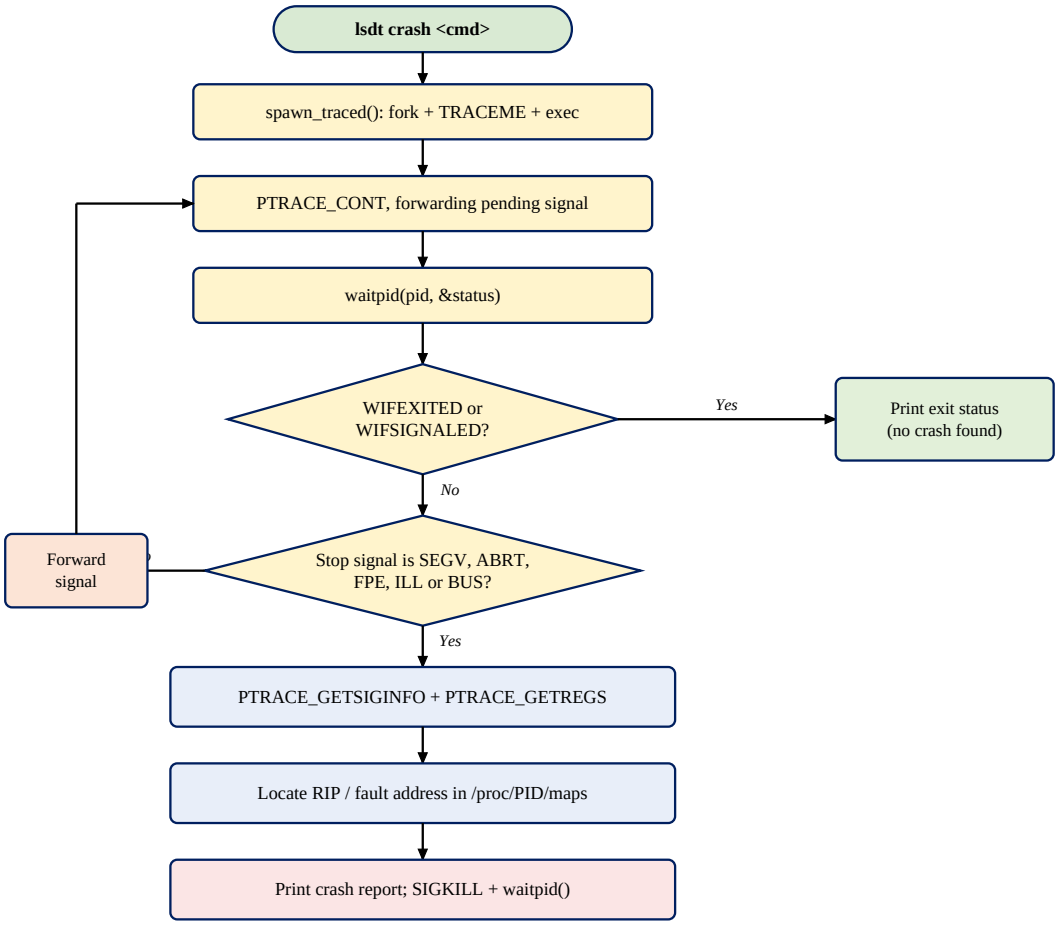


Figure 5.4: Flowchart of the crash analyser (`lsdt crash`)

5.3.3 System-call tracing sequence

The interaction between the tracer and the traced program is a sequence of alternating resume and wait operations (Figure 5.5). For every system call the kernel stops the program twice: once on entry, when the call number and arguments are available in the registers, and once on exit, when the return value is available in RAX. The tool measures the time between the two stops to compute the duration of the call.

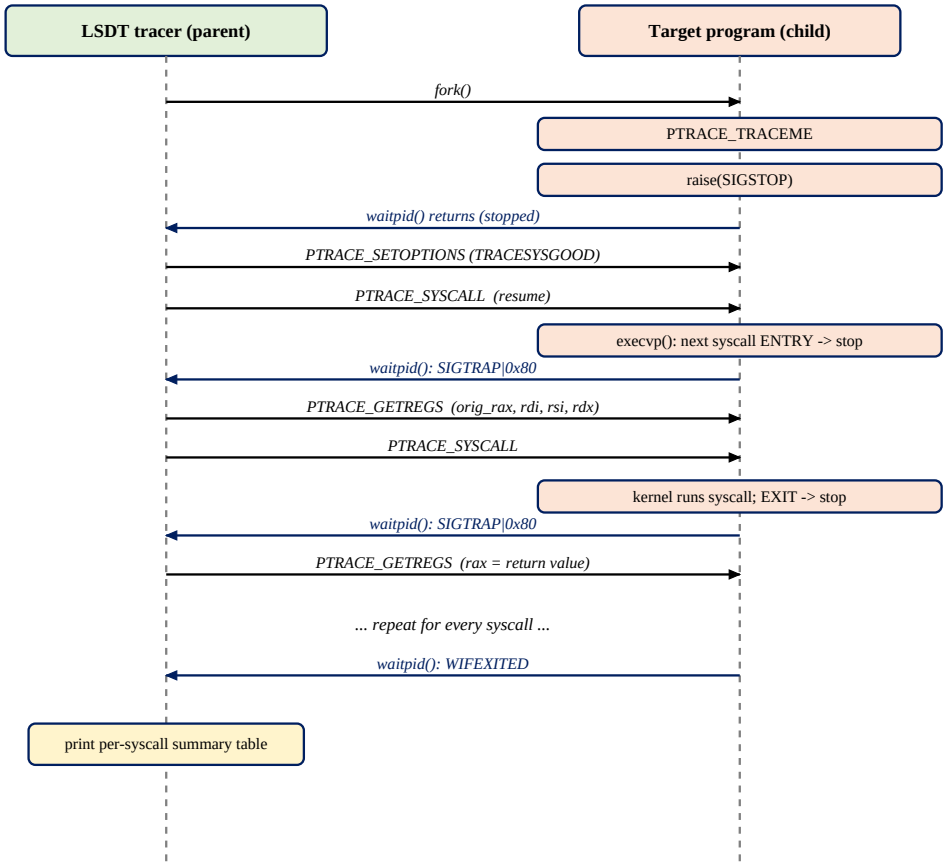


Figure 5.5: Sequence of interactions between the LSDT tracer and the traced program

5.4 Module Description

No.	Module	Command	Kernel interface	Main output
1	Process Inspector	`info PID`	/proc/PID/status, cmdline, exe, cwd, limits	Name, state, PPid, UID, threads, VmSize, VmRSS, context switches, paths
2	File Descriptor Inspector	`fds PID`	/proc/PID/fd, stat()	FD, type, target, total count
3	Thread Inspector	`threads PID`	/proc/PID/task/TID/stat	TID, name, state, user / system CPU seconds
4	Memory Map Analyser	`mem PID`	/proc/PID/maps, smaps_rollup	Mapping table, totals by region type, Rss / Pss / Private_Dirty
5	Live Resource Monitor	`watch PID [ms] [n]`	/proc/PID/stat, pipe, pthreads	CPU %, RSS, threads per sample, peaks, CSV log
6	System-Call Tracer	`trace [-c] CMD`	ptrace(PTRACE_SYSCALL), waitpid	Call trace with arguments, result, duration; statistics table
7	Crash Analyser	`crash CMD`	ptrace(CONT, GETSIGINFO, GETREGS), maps	Signal, cause, fault address, registers, region
8	System Overview	`sys`	/proc/loadavg, uptime, meminfo, /proc listing	Load, uptime, memory, process count

5.4.1 Process Inspector

Input: a PID. **Processing:** reads `/proc/PID/status` into a buffer and extracts eleven keys with `status_field()`; reads `cmdline` (arguments separated by NUL characters, which are converted to spaces); resolves `exe` and `cwd` with `readlink`; extracts the "Max open files" line from `limits`. **Output:** a labelled list of fields. **Error handling:** if the PID does not exist, `read_file` fails and the module prints the `perror` message.

5.4.2 File Descriptor Inspector

Input: a PID. **Processing:** opens the directory `/proc/PID/fd`; for each entry other than "." and "..", `readlink` gives the target (a path, `pipe:[inode]`, `socket:[inode]` or `anon_inode:...`) and `stat` through the link determines the object type using the `S_IS*` macros. **Output:** one row per descriptor and a total. This is the module that reveals a descriptor leak: a growing list of identical targets.

5.4.3 Thread Inspector

Input: a PID. **Processing:** for each directory under `/proc/PID/task`, the per-thread `stat` file is read. Because the thread name is enclosed in parentheses and may itself contain spaces or parentheses, the parser locates the **last** closing parenthesis with `strrchr` and reads the remaining fields from there. Fields 3, 14 and 15 are the state, user time and system time in clock ticks, converted to seconds with `sysconf(_SC_CLK_TCK)`. **Output:** one row per thread.

5.4.4 Memory Map Analyser

Input: a PID. **Processing:** every line of `/proc/PID/maps` is parsed with `sscanf` into start address, end address, permissions and path. The size in kilobytes is computed and the region is classified: `[heap]` as heap, `[stack...]` as stack, other bracketed names (`[vdso]`, `[vvar]`) as special, names beginning with "/" as file-backed and everything else as anonymous. Totals per class are accumulated, and `smaps_rollup` is read for Rss, Pss, Shared_Clean, Private_Dirty and Swap. **Output:** the first 25 mappings, the number of remaining mappings, the totals and the memory summary.

5.4.5 Live Resource Monitor

Input: PID, interval in milliseconds (default 500) and number of samples (default 6). **Processing:** see Section 5.2.3 and Section 6.6. **Output:** a table row per sample, a summary line with the number of records produced and consumed (which must be equal) and the peak CPU and peak RSS, and a CSV log file.

5.4.6 System-Call Tracer

Input: a program and its arguments, optionally preceded by `-c`. **Processing:** the toggle between entry and exit, decoding of the call name (`sysname`), reading of file-name arguments for `open`, `openat`, `execve` and `access` using `peek_string`, timing with `clock_gettime`, and accumulation in `struct sysstat tab[512]`. **Output:** in normal mode one line per call in the form `name(args) = result <seconds>`, with `-1 ERRNAME (message)` for failures; in both modes a statistics table with call counts, error counts and time per call, and a total.

5.4.7 Crash Analyser

Input: a program and its arguments. **Processing:** see Figure 5.4. When a fatal signal stop is observed, `PTRACE_GETSIGINFO` yields `si_code` and `si_addr`; `PTRACE_GETREGS` yields RIP, RSP, RBP and the general registers; `describe_addr()` searches `/proc/PID/maps` for the region containing a given address. **Output:** a crash report. A fault address that is not inside any mapping (such as 0x0) is reported as "not mapped".

5.4.8 System Overview

Processing: reads `/proc/loadavg` (1, 5 and 15-minute load averages, running/total tasks, last PID), `/proc/uptime`, selected keys of `/proc/meminfo`, and counts the numeric directory names in `/proc` to obtain the number of processes. **Output:** a short summary of the machine state, which gives context for interpreting the other modules.

5.5 Input and Output Design

5.5.1 Command-line grammar

```
lsdt  <command>  [arguments]

command  ::=  "sys"
           |  "info"    PID
           |  "fds"     PID
           |  "threads" PID
           |  "mem"     PID
           |  "watch"   PID [interval_ms [samples]]      defaults: 500 ms, 6 samples
           |  "trace"   [ "-c" ] program [program-args]  -c = statistics only
           |  "crash"   program [program-args]
```

5.5.2 Inputs

Input	Type	Validation
PID	Positive integer	Converted with `atoi`; a non-existent PID causes the failing `open` to be reported with `perror`
interval_ms	Integer milliseconds	Optional; default 500
samples	Integer	Optional; default 6
program, program-args	Path or name resolved through PATH	`execvp` failure prints the error and the child exits with status 127

5.5.3 Outputs and exit codes

All reports are written to standard output. Each report begins with a title line of the form `=== TITLE : PID n ===`, followed by two-space indented fields or aligned columns, so that reports are easy to read and easy to process with `grep` and `awk`. Messages from the tracer itself (the "+++ exited with N +++" line) go to standard error so they can be separated from the target program's output.

Exit code	Meaning
0	The command completed successfully
1	Wrong or missing arguments, or the process/file could not be accessed
2	(`crash` only) A crash was detected in the target program
127	(child process) The target program could not be executed

5.5.4 Log file format

The live monitor writes `lsdt_monitor.log` in comma-separated form, one line per sample with the columns **sample number, CPU percentage, resident memory in kilobytes, number of threads**. The file can be opened directly in a spreadsheet or plotted with Python; the chart in Figure 7.16 was produced from such a log.

```
1,92.0,10888,4
2,94.6,15496,4
3,97.5,18568,4
```

5.5.5 Design of the messages

- Error messages follow the UNIX convention: `perror("what failed")` prints the operation followed by the system's own error text.
- Failed system calls in the trace are printed in the same style as strace: `= -1 ENOENT (No such file or directory)`.
- Numbers are printed in the form that the reader expects: addresses in hexadecimal, sizes in kilobytes, times in seconds with six decimals, CPU in percent with one decimal.
- The help text printed by running `lsdt` with no argument lists every command with a one-line description.

CHAPTER - 6
IMPLEMENTATION

6.1 Module-wise Implementation

This section explains how each of the eight modules was built, in the order in which the team developed them. The complete source code is `lsdt.c`; the functions are listed in Section 6.3. Every module follows the same pattern: **collect** (read from the kernel), **analyse** (parse, convert, accumulate), **present** (print).

6.1.1 Common helper functions

- `read_file(path, buf, n)` opens a file, reads at most `n-1` bytes and appends a terminating NUL so that the buffer can be handled as a C string. It returns the number of bytes read, or -1 on any failure.
- `status_field(text, key, out, n)` scans a buffer that contains lines of the form `Key:<TAB>value` and copies the value of the requested key into `out`, never writing more than `n` bytes. It is used for /proc/PID/status, smaps_rollup and meminfo, which share this format.
- `now_sec()` returns the monotonic clock as a `double` in seconds. The monotonic clock is not affected by changes of the system time, which makes it correct for measuring durations.
- `die(msg)` prints a `perror` message and terminates; it is used only for unrecoverable failures such as `fork` or `pipe`.

6.1.2 Process Inspector (`cmd_info`)

1. Build `/proc/PID/status` and read it into an 8 KB buffer; stop with an error message if the process does not exist.
2. Loop over an array of eleven key names and print each value found with `status_field`.
3. Read `cmdline`, replace each NUL separator with a blank and print it.
4. Use `readlink` on `/proc/PID/exe` and `/proc/PID/cwd`.
5. Read `/proc/PID/limits`, find the "Max open files" line with `strstr` and print it.

6.1.3 File Descriptor and Thread Inspectors (`cmd_fds`, `cmd_threads`)

Both modules enumerate a directory with `opendir` / `readdir`, skip the entries that start with a dot, and handle one entry at a time. `cmd_fds` resolves each link with `readlink` and determines the type with `stat`. `cmd_threads` reads the `stat` file of each thread. A difficulty encountered here was that the thread name is stored between parentheses and **may contain spaces or even parentheses** (for example a process renamed with `prctl`). Splitting the line on blanks would then shift every following field, so the implementation finds the **last** `)` with `strrchr` and parses the numeric fields only after it.

6.1.4 Memory Map Analyser (`cmd_mem`)

Each line of `/proc/PID/maps` looks like `7f99c1628000-7f99c17b0000 r-xp 00028000 fe:00 573510 /usr/lib/x86_64-linux-gnu/libc.so.6`. The line is parsed with one `sscanf` call whose format skips the offset, device and inode fields (`%*s`) and captures everything to the end of the line as the name (`%299[^\n]`). The name decides the classification. Counters of kilobytes per class are kept in `unsigned long` variables so that very large address spaces (hundreds of gigabytes of virtual space) cannot overflow. Only the first 25 mappings are printed to keep the report readable; the totals always include all mappings.

6.1.5 System-Call Tracer (`cmd_trace`)

The tracer is built on `spawn_traced()` (fork, `PTRACE_TRACEME`, `raise(SIGSTOP)`, `execvp`). In the main loop the variable `in_call` flips at every syscall stop. At **entry** the call number is read from `orig_rax` (the original value of RAX, which is preserved by the kernel when RAX is later overwritten by the return value) and the start time is recorded; for `open`, `openat`, `execve` and `access` the pointer argument is dereferenced in the tracee's address space with `peek_string` to print the file name. At **exit** the return value is read from `rax`; a value between -4095 and -1 is treated as an error and decoded with `strerrorname_np` (symbolic name, for example ENOENT) and `strerror` (message). The duration, count and error count are accumulated in a table indexed by syscall number, which is printed when the child exits.

6.1.6 Crash Analyser (`cmd_crash`)

The crash analyser reuses `spawn_traced()` but resumes the child with `PTRACE_CONT`. After each stop, `WSTOPSIG` gives the signal that stopped the child. If it is one of SIGSEGV, SIGABRT, SIGFPE, SIGILL or SIGBUS, the report is generated; every other signal is passed back to the child on the next `PTRACE_CONT` (the `sig` variable), so that normal signal handling by the program is not disturbed. After printing, the child is killed with SIGKILL and reaped with `waitpid` so that no zombie remains.

6.1.7 Live Resource Monitor (`cmd_watch`)

The monitor creates a pipe, starts the logger and sampler threads and waits for both. The sampler computes CPU percentage as the change in `utime + stime` (in jiffies) divided by `CLK_TCK` and by the elapsed monotonic time, and converts the resident page count to kilobytes by multiplying by the page size. Section 6.6 describes the synchronisation.

6.1.8 System Overview (`cmd_sys`)

The overview reads three /proc files and counts the directory entries of /proc whose names begin with a digit 1 to 9 (each is a process). It is intentionally the simplest module and is useful as the first command a user runs.

6.1.9 Problems encountered and how they were solved

Problem	Observation	Solution
Format-truncation warnings	GCC warned that a 128-byte path buffer could be too small	Buffers enlarged to 512 bytes; build is now warning-free
Interleaved output	When the output was redirected to a file or pipe, the tracer's lines and the traced program's output were written in the wrong order because the tool's stdout was fully buffered while the child's output was not	Call `setvbuf(stdout, NULL, _IOLBF, 0)` at start so that each line is flushed immediately
Parsing thread names	Names with spaces break blank-separated parsing	Find the last `)` with `strrchr`, parse after it
Reading strings from the tracee	`PTRACE_PEEKDATA` returns the data word itself, so -1 is a legitimate value and cannot signal an error	Set `errno = 0` before the call and test `errno` afterwards
Distinguishing syscall stops	An ordinary SIGTRAP could be mistaken for a syscall stop	Use `PTRACE_O_TRACESYSGOOD` so that syscall stops carry `SIGTRAP | 0x80`
Unknown syscall numbers	The kernel has more than 400 calls; naming all of them is unnecessary	`sysname()` names the commonly seen ones and falls back to `syscall_N`; statistics are still correct for all
Pointer return values	Decimal values of mmap/brk results are unreadable	Print results of `mmap` and `brk` in hexadecimal

6.2 Algorithm / Pseudocode

Algorithm 1: Process inspection

```
INFO(pid):
    text <- READ("/proc/<pid>/status")        # fail -> print error, return 1
    FOR key IN [Name, State, PPid, Uid, Threads, VmSize, VmRSS, VmData,
                VmStk, voluntary_ctxt_switches, nonvoluntary_ctxt_switches]:
        PRINT key, FIELD(text, key)
    cmd  <- READ("/proc/<pid>/cmdline");  replace NUL by blank;  PRINT cmd
    PRINT READLINK("/proc/<pid>/exe"), READLINK("/proc/<pid>/cwd")
    PRINT line "Max open files" of READ("/proc/<pid>/limits")
```

Algorithm 2: Memory-map classification

```
MEM(pid):
    heap = stack = anon = file = other = 0
    FOR EACH line IN "/proc/<pid>/maps":
        (lo, hi, perm, name) <- PARSE(line);   kb <- (hi - lo) / 1024
        IF   name == "[heap]"          THEN heap  += kb
        ELIF name STARTS WITH "[stack" THEN stack += kb
        ELIF name STARTS WITH "["      THEN other += kb     # vdso, vvar
        ELIF name STARTS WITH "/"      THEN file  += kb     # exe, libraries
        ELSE                                anon  += kb
        IF count <= 25 THEN PRINT lo-hi, perm, kb, name
    PRINT mapping count, heap, stack, file, anon, other
    PRINT Rss, Pss, Shared_Clean, Private_Dirty, Swap FROM smaps_rollup
```

Algorithm 3: System-call tracing

```
TRACE(program, summary_only):
    pid <- SPAWN_TRACED(program)        # fork; child: TRACEME, SIGSTOP, exec
    in_call <- FALSE;  table[0..511] <- zeros
    LOOP:
        PTRACE_SYSCALL(pid);  status <- WAITPID(pid)
        IF EXITED(status) or SIGNALED(status)  THEN BREAK
        IF status is not (SIGTRAP | 0x80)      THEN CONTINUE
        regs <- PTRACE_GETREGS(pid)
        IF in_call == FALSE THEN                    # ENTRY
            in_call <- TRUE;  nr <- regs.orig_rax;  t0 <- NOW()
            IF NOT summary_only THEN  text <- NAME(nr) + "(" + ARGS(nr, regs) + ")"
        ELSE                                         # EXIT
            in_call <- FALSE;  dt <- NOW() - t0;  rv <- regs.rax
            table[nr].calls++;  table[nr].secs += dt
            IF -4096 < rv < 0 THEN table[nr].errors++
            IF NOT summary_only THEN PRINT text, rv or errno name, dt
    PRINT table (calls, errors, time per syscall) and total
```

Algorithm 4: Crash analysis

```
CRASH(program):
    pid <- SPAWN_TRACED(program);   sig <- 0
    LOOP:
        PTRACE_CONT(pid, sig);  sig <- 0;  status <- WAITPID(pid)
        IF EXITED(status)   THEN PRINT "no crash", RETURN 0
        IF SIGNALED(status) THEN PRINT "terminated by signal", RETURN 1
        s <- STOPSIG(status)
        IF s IN {SIGSEGV, SIGABRT, SIGFPE, SIGILL, SIGBUS} THEN
            info <- PTRACE_GETSIGINFO(pid);  regs <- PTRACE_GETREGS(pid)
            PRINT signal name, info.si_code (decoded), info.si_addr
            PRINT RIP, RSP, RBP, RAX, RBX, RCX, RDX, RSI, RDI, EFLAGS
            PRINT REGION_OF(regs.rip);  IF s == SIGSEGV THEN PRINT REGION_OF(info.si_addr)
            KILL(pid, SIGKILL);  WAITPID(pid);  RETURN 2
        ELIF s NOT IN {SIGTRAP, SIGSTOP} THEN sig <- s     # forward to program
```

Algorithm 5: Live monitor with producer-consumer

```
WATCH(pid, interval, n):
    (rfd, wfd) <- PIPE();   m <- {produced=0, consumed=0, peaks=0, lock}
    START logger_thread(m);  START sampler_thread(m);  JOIN both;  PRINT summary

sampler_thread (producer):
    (prev_jiffies, ..) <- READ_STAT(pid);  tprev <- NOW()
    REPEAT n TIMES:
        SLEEP(interval);   (jiffies, rss_pages, threads) <- READ_STAT(pid)
        t <- NOW();  cpu <- 100 * ((jiffies - prev)/CLK_TCK) / (t - tprev)
        WRITE(wfd, "i,cpu,rss_kb,threads\n")
        LOCK(m);  m.produced++;  UPDATE peaks;  UNLOCK(m)
        prev <- jiffies;  tprev <- t
    CLOSE(wfd)                                  # signals end-of-data

logger_thread (consumer):
    WHILE line <- FGETS(rfd) is not EOF:
        PRINT row;  APPEND line TO log file
        LOCK(m);  m.consumed++;  UNLOCK(m)
```

Algorithm 6: CPU utilisation from /proc/PID/stat

```
cpu_percent = 100 x ( (utime + stime)_now - (utime + stime)_before ) / CLK_TCK
              ------------------------------------------------------------
                          (t_now - t_before)   [seconds]

Example (400 ms interval, CLK_TCK = 100): the process used 38 more ticks
    -> 0.38 s of CPU in 0.40 s of wall time  ->  95 %
```

6.3 Core C / C++ Implementation

The listings below are taken directly from `lsdt.c` (607 lines in total). The tool is written in C; no C++ is required. The program starts with the include section and the common helper functions.

6.3.1 Header, includes and helper functions

```
/*
 * LSDT - Linux System Debugging Tool
 * K L (Deemed to be) University, Dept. of CSE
 * Team: 2520030012 Dharmika | 2520030501 Anushka K | 2520030542 Rushvini
 *
 * Build : gcc -Wall -Wextra -O2 -pthread -o lsdt lsdt.c
 * Usage : ./lsdt <command> [args]   (see usage() below)
 */
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <signal.h>
#include <dirent.h>
#include <pthread.h>
#include <time.h>
#include <fcntl.h>
#include <sys/ptrace.h>
#include <sys/wait.h>
#include <sys/user.h>
#include <sys/types.h>
#include <sys/stat.h>

/* ======================= common helpers ======================= */

static void die(const char *msg) { perror(msg); exit(EXIT_FAILURE); }

static int read_file(const char *path, char *buf, size_t n)
{
    int fd = open(path, O_RDONLY);
    if (fd < 0) return -1;
    ssize_t r = read(fd, buf, n - 1);
    close(fd);
    if (r < 0) return -1;
    buf[r] = '\0';
    return (int)r;
}

static double now_sec(void)
{
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return ts.tv_sec + ts.tv_nsec / 1e9;
}

static int status_field(const char *txt, const char *key, char *out, size_t n)
{
    size_t kl = strlen(key);
    for (const char *p = txt; p && *p; p = strchr(p, '\n') ? strchr(p, '\n') + 1 : NULL) {
        if (strncmp(p, key, kl) == 0 && p[kl] == ':') {
            p += kl + 1;
            while (*p == ' ' || *p == '\t') p++;
            size_t i = 0;
            while (*p && *p != '\n' && i < n - 1) out[i++] = *p++;
            out[i] = '\0';
            return 0;
        }
    }
    out[0] = '\0';
    return -1;
}
```

`status_field` walks the buffer line by line (the `for` loop advances to the character after the next newline), compares the beginning of the line with the key followed by a colon and copies the value, skipping the spaces or tabs that follow the colon. It writes a terminating NUL in every case, so the

caller can always print the result.

6.3.2 Process Inspector

```
static int cmd_info(pid_t pid)
{
    char path[64], buf[8192], v[256];
    snprintf(path, sizeof path, "/proc/%d/status", pid);
    if (read_file(path, buf, sizeof buf) < 0) { perror("status"); return 1; }

    printf("=== PROCESS INSPECTOR : PID %d ===\n", pid);
    const char *keys[] = { "Name", "State", "PPid", "Uid", "Threads",
                           "VmSize", "VmRSS", "VmData", "VmStk",
                           "voluntary_ctxt_switches",
                           "nonvoluntary_ctxt_switches" };
    for (size_t i = 0; i < sizeof keys / sizeof *keys; i++)
        if (status_field(buf, keys[i], v, sizeof v) == 0)
            printf("  %-28s %s\n", keys[i], v);

    snprintf(path, sizeof path, "/proc/%d/cmdline", pid);
    int n = read_file(path, buf, sizeof buf);
    for (int i = 0; i < n; i++) if (buf[i] == '\0') buf[i] = ' ';
    printf("  %-28s %s\n", "Command line", n > 0 ? buf : "(kernel thread)");

    snprintf(path, sizeof path, "/proc/%d/exe", pid);
    ssize_t l = readlink(path, v, sizeof v - 1);
    if (l > 0) { v[l] = '\0'; printf("  %-28s %s\n", "Executable", v); }
    snprintf(path, sizeof path, "/proc/%d/cwd", pid);
    l = readlink(path, v, sizeof v - 1);
    if (l > 0) { v[l] = '\0'; printf("  %-28s %s\n", "Working directory", v); }

    snprintf(path, sizeof path, "/proc/%d/limits", pid);
    if (read_file(path, buf, sizeof buf) > 0) {
        char *line = strstr(buf, "Max open files");
        if (line) { char *e = strchr(line, '\n'); if (e) *e = 0; printf("  %s\n", line); }
    }
    return 0;
}
```

6.3.3 System-call tracer

The function `sysname` is a `switch` statement that maps the numbers of about forty common x86_64 system calls to their names (for example 0 to `read`, 1 to `write`, 257 to `openat`, 231 to `exit_group`); it is not repeated here. The remaining tracer code is:

```
static void peek_string(pid_t pid, unsigned long addr, char *out, size_t n)
{
    size_t i = 0;
    while (i + 1 < n) {
        errno = 0;
        long w = ptrace(PTRACE_PEEKDATA, pid, (void *)(addr + i), NULL);
        if (errno) break;
        for (size_t b = 0; b < sizeof w && i + 1 < n; b++) {
            char c = (char)((w >> (8 * b)) & 0xff);
            if (c == '\0') { out[i] = '\0'; return; }
            out[i++] = c;
        }
    }
    out[i] = '\0';
}

static pid_t spawn_traced(char **argv)
{
    pid_t pid = fork();
    if (pid < 0) die("fork");
    if (pid == 0) {
        if (ptrace(PTRACE_TRACEME, 0, NULL, NULL) < 0) die("TRACEME");
        raise(SIGSTOP);
        execvp(argv[0], argv);
        perror("execvp");
        _exit(127);
    }
    int st;
    waitpid(pid, &st, 0);                       /* child stopped by SIGSTOP */
    ptrace(PTRACE_SETOPTIONS, pid, NULL, (void *)PTRACE_O_TRACESYSGOOD);
    return pid;
```

```
}

static int cmd_trace(char **argv, int summary_only)
{
    struct sysstat tab[MAXSYS];
    memset(tab, 0, sizeof tab);
    pid_t pid = spawn_traced(argv);
    int status, in_call = 0;
    long cur_nr = -1;
    double t0 = 0;
    char path[256], line[512];
    long total = 0;

    for (;;) {
        ptrace(PTRACE_SYSCALL, pid, NULL, NULL);
        waitpid(pid, &status, 0);
        if (WIFEXITED(status)) {
            fprintf(stderr, "+++ exited with %d +++\n", WEXITSTATUS(status));
            break;
        }
        if (WIFSIGNALED(status)) {
            fprintf(stderr, "+++ killed by signal %d +++\n", WTERMSIG(status));
            break;
        }
        if (!(WIFSTOPPED(status) && WSTOPSIG(status) == (SIGTRAP | 0x80)))
            continue;

        struct user_regs_struct r;
        ptrace(PTRACE_GETREGS, pid, NULL, &r);
        if (!in_call) {                         /* syscall ENTRY */
            in_call = 1;
            cur_nr = (long)r.orig_rax;
            t0 = now_sec();
            if (!summary_only) {
                const char *nm = sysname(cur_nr);
                char a[64];
                if (cur_nr == 2 || cur_nr == 257 || cur_nr == 59 || cur_nr == 21) {
                    unsigned long p = (cur_nr == 257) ? r.rsi : r.rdi;
                    peek_string(pid, p, path, sizeof path);
                    snprintf(a, sizeof a, "\"%.40s\"", path);
                } else {
                    snprintf(a, sizeof a, "0x%llx, 0x%llx, 0x%llx",
                             r.rdi, r.rsi, r.rdx);
                }
                if (nm) snprintf(line, sizeof line, "%s(%s)", nm, a);
                else    snprintf(line, sizeof line, "syscall_%ld(%s)", cur_nr, a);
            }
        } else {                                /* syscall EXIT  */
            in_call = 0;
            double dt = now_sec() - t0;
            long rv = (long)r.rax;
            if (cur_nr >= 0 && cur_nr < MAXSYS) {
                tab[cur_nr].calls++;
                tab[cur_nr].secs += dt;
                if (rv < 0 && rv > -4096) tab[cur_nr].errors++;
            }
            total++;
            if (!summary_only) {
                if (rv < 0 && rv > -4096)
                    printf("%-52s = -1 %s (%s)  <%.6f>\n", line,
                           strerrorname_np(-rv), strerror(-rv), dt);
                else if (cur_nr == 9 || cur_nr == 12)
                    printf("%-52s = 0x%lx  <%.6f>\n", line, rv, dt);
                else
                    printf("%-52s = %ld  <%.6f>\n", line, rv, dt);
            }
        }
    }

    printf("\n%-18s %8s %8s %12s\n", "SYSCALL", "CALLS", "ERRORS", "TIME(s)");
    printf("------------------ -------- -------- ------------\n");
    for (int i = 0; i < MAXSYS; i++) {
        if (!tab[i].calls) continue;
        const char *nm = sysname(i);
        char tmp[24];
        if (!nm) { snprintf(tmp, sizeof tmp, "syscall_%d", i); nm = tmp; }
        printf("%-18s %8ld %8ld %12.6f\n", nm, tab[i].calls,
               tab[i].errors, tab[i].secs);
    }
    printf("------------------ -------- -------- ------------\n");
```

```
    printf("%-18s %8ld\n", "TOTAL", total);
    return 0;
}
```

The key idea is in the two branches of `if (!in_call)`. At entry, `r.orig_rax` holds the call number and `r.rdi`, `r.rsi`, `r.rdx` hold the first three arguments. At exit, `r.rax` holds the result. The difference of the two `now_sec()` values is the time spent inside the kernel (plus the tracing overhead).

6.3.4 Memory Map Analyser

```
static int cmd_mem(pid_t pid)
{
    char path[64], line[512];
    snprintf(path, sizeof path, "/proc/%d/maps", pid);
    FILE *f = fopen(path, "r");
    if (!f) { perror("maps"); return 1; }

    unsigned long tot_heap = 0, tot_stack = 0, tot_anon = 0, tot_file = 0, tot_other = 0;
    int nmap = 0;
    printf("=== MEMORY MAP ANALYSER : PID %d ===\n", pid);
    printf("%-33s %-5s %9s  %s\n", "ADDRESS RANGE", "PERM", "SIZE(KB)", "MAPPING");
    while (fgets(line, sizeof line, f)) {
        unsigned long lo, hi;
        char perm[8], name[300] = "";
        if (sscanf(line, "%lx-%lx %7s %*s %*s %*s %299[^\n]", &lo, &hi, perm, name) < 3)
            continue;
        char *nm = name;
        while (*nm == ' ') nm++;
        unsigned long kb = (hi - lo) / 1024;
        const char *kind;
        if (!strcmp(nm, "[heap]"))                { kind = "HEAP";  tot_heap  += kb; }
        else if (!strncmp(nm, "[stack", 6))       { kind = "STACK"; tot_stack += kb; }
        else if (nm[0] == '[')                    { kind = nm;      tot_other += kb; }
        else if (nm[0] == '/')                    { kind = nm;      tot_file  += kb; }
        else                                      { kind = "[anon]"; tot_anon += kb; }
        if (nmap++ < 25)
            printf("%08lx-%08lx  %-5s %9lu  %s\n", lo, hi, perm, kb, kind);
    }
    fclose(f);
    if (nmap > 25) printf("... (%d more mappings)\n", nmap - 25);

    printf("\n  Total mappings        : %d\n", nmap);
    printf("  Heap                  : %lu KB\n", tot_heap);
    printf("  Stack                 : %lu KB\n", tot_stack);
    printf("  File-backed (libs/exe): %lu KB\n", tot_file);
    printf("  Anonymous             : %lu KB\n", tot_anon);
    printf("  Special (vdso etc.)   : %lu KB\n", tot_other);

    char buf[2048], v[64];
    snprintf(path, sizeof path, "/proc/%d/smaps_rollup", pid);
    if (read_file(path, buf, sizeof buf) > 0) {
        printf("\n  -- smaps_rollup --\n");
        const char *k[] = { "Rss", "Pss", "Shared_Clean", "Private_Dirty", "Swap" };
        for (size_t i = 0; i < 5; i++)
            if (status_field(buf, k[i], v, sizeof v) == 0)
                printf("  %-16s %s\n", k[i], v);
    }
    return 0;
}
```

6.3.5 File Descriptor and Thread Inspectors

```
static int cmd_fds(pid_t pid)
{
    char dir[64], p[512], target[512];
    snprintf(dir, sizeof dir, "/proc/%d/fd", pid);
    DIR *d = opendir(dir);
    if (!d) { perror("fd"); return 1; }
    printf("=== OPEN FILE DESCRIPTORS : PID %d ===\n", pid);
    printf("%-5s %-8s %s\n", "FD", "TYPE", "TARGET");
    struct dirent *e;
    int count = 0;
    while ((e = readdir(d))) {
        if (e->d_name[0] == '.') continue;
        snprintf(p, sizeof p, "%s/%s", dir, e->d_name);
```

```
        ssize_t l = readlink(p, target, sizeof target - 1);
        if (l < 0) continue;
        target[l] = '\0';
        struct stat st;
        const char *type = "other";
        if (stat(p, &st) == 0) {
            if (S_ISREG(st.st_mode)) type = "FILE";
            else if (S_ISDIR(st.st_mode)) type = "DIR";
            else if (S_ISCHR(st.st_mode)) type = "CHAR";
            else if (S_ISFIFO(st.st_mode)) type = "PIPE";
            else if (S_ISSOCK(st.st_mode)) type = "SOCKET";
        }
        printf("%-5s %-8s %s\n", e->d_name, type, target);
        count++;
    }
    closedir(d);
    printf("Total open descriptors: %d\n", count);
    return 0;
}

static int cmd_threads(pid_t pid)
{
    char dir[64], p[512], buf[1024];
    snprintf(dir, sizeof dir, "/proc/%d/task", pid);
    DIR *d = opendir(dir);
    if (!d) { perror("task"); return 1; }
    long hz = sysconf(_SC_CLK_TCK);
    printf("=== THREAD INSPECTOR : PID %d ===\n", pid);
    printf("%-8s %-16s %-6s %10s %10s\n", "TID", "NAME", "STATE", "UTIME(s)", "STIME(s)");
    struct dirent *e;
    while ((e = readdir(d))) {
        if (e->d_name[0] == '.') continue;
        snprintf(p, sizeof p, "%s/%s/stat", dir, e->d_name);
        if (read_file(p, buf, sizeof buf) < 0) continue;
        char *lp = strchr(buf, '('), *rp = strrchr(buf, ')');
        if (!lp || !rp) continue;
        *rp = '\0';
        char state; unsigned long ut, st;
        sscanf(rp + 2, "%c %*d %*d %*d %*d %*d %*u %*u %*u %*u %*u %lu %lu",
               &state, &ut, &st);
        printf("%-8s %-16s %-6c %10.2f %10.2f\n", e->d_name, lp + 1, state,
               (double)ut / hz, (double)st / hz);
    }
    closedir(d);
    return 0;
}
```

6.3.6 Crash Analyser

```
static const char *signame(int s)
{
    switch (s) {
    case SIGSEGV: return "SIGSEGV (invalid memory reference)";
    case SIGABRT: return "SIGABRT (abort)";
    case SIGFPE:  return "SIGFPE (arithmetic exception)";
    case SIGILL:  return "SIGILL (illegal instruction)";
    case SIGBUS:  return "SIGBUS (bus error)";
    default:      return "other";
    }
}

static const char *code_text(int sig, int code)
{
    if (sig == SIGSEGV && code == SEGV_MAPERR) return " (SEGV_MAPERR: address not mapped)";
    if (sig == SIGSEGV && code == SEGV_ACCERR) return " (SEGV_ACCERR: permission denied)";
    if (sig == SIGFPE  && code == FPE_INTDIV)  return " (FPE_INTDIV: integer divide by zero)";
    return "";
}

static void describe_addr(pid_t pid, unsigned long addr)
{
    char path[64], line[512];
    snprintf(path, sizeof path, "/proc/%d/maps", pid);
    FILE *f = fopen(path, "r");
    if (!f) return;
    while (fgets(line, sizeof line, f)) {
        unsigned long lo, hi;
        char perm[8], name[300] = "";
        if (sscanf(line, "%lx-%lx %7s %*s %*s %*s %299[^\n]", &lo, &hi, perm, name) >= 3
```

```
            && addr >= lo && addr < hi) {
            char *nm = name;
            while (*nm == ' ') nm++;
            printf("  Region        : %lx-%lx %s %s\n", lo, hi, perm, *nm ? nm : "[anon]");
            fclose(f);
            return;
        }
    }
    fclose(f);
    printf("  Region        : not mapped (unmapped / NULL page access)\n");
}

static int cmd_crash(char **argv)
{
    pid_t pid = spawn_traced(argv);
    int status, sig = 0;
    for (;;) {
        ptrace(PTRACE_CONT, pid, NULL, (void *)(long)sig);
        sig = 0;
        waitpid(pid, &status, 0);
        if (WIFEXITED(status)) {
            printf("Target exited normally with status %d. No crash detected.\n",
                   WEXITSTATUS(status));
            return 0;
        }
        if (WIFSIGNALED(status)) {
            printf("Target terminated by signal %d.\n", WTERMSIG(status));
            return 1;
        }
        int s = WSTOPSIG(status);
        if (s == SIGSEGV || s == SIGABRT || s == SIGFPE || s == SIGILL || s == SIGBUS) {
            siginfo_t si;
            struct user_regs_struct r;
            ptrace(PTRACE_GETSIGINFO, pid, NULL, &si);
            ptrace(PTRACE_GETREGS, pid, NULL, &r);
            printf("=== CRASH REPORT : PID %d ===\n", pid);
            printf("  Signal        : %s\n", signame(s));
            printf("  si_code       : %d%s\n", si.si_code, code_text(s, si.si_code));
            if (s == SIGSEGV || s == SIGBUS || s == SIGFPE || s == SIGILL)
                printf("  Fault address : %p\n", si.si_addr);
            printf("  RIP (instr.)  : 0x%llx\n", r.rip);
            printf("  RSP (stack)   : 0x%llx\n", r.rsp);
            printf("  RBP           : 0x%llx\n", r.rbp);
            printf("  RAX 0x%llx  RBX 0x%llx  RCX 0x%llx  RDX 0x%llx\n",
                   r.rax, r.rbx, r.rcx, r.rdx);
            printf("  RSI 0x%llx  RDI 0x%llx  EFLAGS 0x%llx\n", r.rsi, r.rdi, r.eflags);
            printf("  --- faulting instruction location ---\n");
            describe_addr(pid, r.rip);
            if (s == SIGSEGV) {
                printf("  --- faulting data address ---\n");
                describe_addr(pid, (unsigned long)si.si_addr);
            }
            kill(pid, SIGKILL);
            waitpid(pid, &status, 0);
            return 2;
        }
        if (s != SIGTRAP && s != SIGSTOP) sig = s;   /* forward other signals */
    }
}
```

Note how the loop works: `PTRACE_CONT` resumes the child; `waitpid` blocks until it stops again. A stop with a fatal signal reaches the report block; any other signal (except the SIGTRAP and SIGSTOP generated by the tracing mechanism itself) is stored in `sig` and delivered on the next `PTRACE_CONT`.

6.3.7 Live Resource Monitor

```
static int read_proc_stat(pid_t pid, unsigned long *jiffies, long *rss_pages, int *thr)
{
    char path[64], buf[1024];
    snprintf(path, sizeof path, "/proc/%d/stat", pid);
    if (read_file(path, buf, sizeof buf) < 0) return -1;
    char *rp = strrchr(buf, ')');
    if (!rp) return -1;
    unsigned long ut, st; long nt, rss;
    if (sscanf(rp + 2, "%*c %*d %*d %*d %*d %*d %*u %*u %*u %*u %*u %lu %lu "
                       "%*d %*d %*d %*d %ld %*d %*u %*u %ld",
               &ut, &st, &nt, &rss) != 4) return -1;
```

```
    *jiffies = ut + st; *rss_pages = rss; *thr = (int)nt;
    return 0;
}

static void *sampler_thread(void *arg)            /* PRODUCER */
{
    struct monitor *m = arg;
    long hz = sysconf(_SC_CLK_TCK), pg = sysconf(_SC_PAGESIZE);
    unsigned long prev = 0, cur; long rss; int thr;
    double tprev = now_sec();
    read_proc_stat(m->pid, &prev, &rss, &thr);
    for (int i = 0; i < m->samples; i++) {
        usleep(m->interval_ms * 1000);
        if (read_proc_stat(m->pid, &cur, &rss, &thr) < 0) break;
        double t = now_sec();
        double cpu = 100.0 * ((double)(cur - prev) / hz) / (t - tprev);
        long rss_kb = rss * pg / 1024;
        char rec[128];
        int n = snprintf(rec, sizeof rec, "%d,%.1f,%ld,%d\n", i + 1, cpu, rss_kb, thr);
        if (write(m->wfd, rec, n) < 0) break;       /* IPC: pipe */
        pthread_mutex_lock(&m->lock);               /* shared stats */
        m->produced++;
        if (cpu > m->peak_cpu) m->peak_cpu = cpu;
        if (rss_kb > m->peak_rss_kb) m->peak_rss_kb = rss_kb;
        pthread_mutex_unlock(&m->lock);
        prev = cur; tprev = t;
    }
    close(m->wfd);                                  /* EOF tells consumer to stop */
    return NULL;
}

static void *logger_thread(void *arg)             /* CONSUMER */
{
    struct monitor *m = arg;
    FILE *in = fdopen(m->rfd, "r");
    FILE *log = fopen("lsdt_monitor.log", "w");
    char line[128];
    printf("%-6s %8s %12s %8s\n", "SAMPLE", "CPU(%)", "RSS(KB)", "THREADS");
    while (fgets(line, sizeof line, in)) {
        int s, t; double cpu; long rss;
        if (sscanf(line, "%d,%lf,%ld,%d", &s, &cpu, &rss, &t) == 4) {
            printf("%-6d %8.1f %12ld %8d\n", s, cpu, rss, t);
            if (log) fputs(line, log);
            pthread_mutex_lock(&m->lock);
            m->consumed++;
            pthread_mutex_unlock(&m->lock);
        }
    }
    if (log) fclose(log);
    fclose(in);
    return NULL;
}

static int cmd_watch(pid_t pid, int interval_ms, int samples)
{
    int fds[2];
    if (pipe(fds) < 0) die("pipe");
    struct monitor m = { .pid = pid, .interval_ms = interval_ms, .samples = samples,
                         .rfd = fds[0], .wfd = fds[1],
                         .lock = PTHREAD_MUTEX_INITIALIZER };
    printf("=== LIVE RESOURCE MONITOR : PID %d (every %d ms) ===\n", pid, interval_ms);
    pthread_t a, b;
    pthread_create(&b, NULL, logger_thread, &m);
    pthread_create(&a, NULL, sampler_thread, &m);
    pthread_join(a, NULL);
    pthread_join(b, NULL);
    printf("\nSamples produced=%ld consumed=%ld  Peak CPU=%.1f%%  Peak RSS=%ld KB\n",
           m.produced, m.consumed, m.peak_cpu, m.peak_rss_kb);
    printf("Log written to lsdt_monitor.log\n");
    return 0;
}
```

6.3.8 System overview, usage and main()

```
static int cmd_sys(void)
{
    char buf[4096], v[128];
    printf("=== SYSTEM OVERVIEW ===\n");
    if (read_file("/proc/loadavg", buf, sizeof buf) > 0) printf("  Load average    : %s", buf);
    if (read_file("/proc/uptime", buf, sizeof buf) > 0) {
        double up; sscanf(buf, "%lf", &up);
        printf("  Uptime          : %.0f s\n", up);
    }
    if (read_file("/proc/meminfo", buf, sizeof buf) > 0) {
        const char *k[] = { "MemTotal", "MemFree", "MemAvailable", "Buffers",
                            "Cached", "SwapTotal", "SwapFree" };
        for (size_t i = 0; i < 7; i++)
            if (status_field(buf, k[i], v, sizeof v) == 0)
                printf("  %-15s : %s\n", k[i], v);
    }
    int procs = 0;
    DIR *d = opendir("/proc");
    struct dirent *e;
    while (d && (e = readdir(d)))
        if (e->d_name[0] >= '1' && e->d_name[0] <= '9') procs++;
    if (d) closedir(d);
    printf("  Running processes: %d\n", procs);
    return 0;
}

static void usage(void)
{
    puts("LSDT - Linux System Debugging Tool\n"
         "Usage:\n"
         "  lsdt sys                       system overview\n"
         "  lsdt info <pid>                process inspector\n"
         "  lsdt fds <pid>                 open file descriptors\n"
         "  lsdt threads <pid>             thread inspector\n"
         "  lsdt mem <pid>                 memory map analyser\n"
         "  lsdt watch <pid> [ms] [n]      live CPU/RSS monitor\n"
         "  lsdt trace [-c] <cmd> [args]   syscall tracer (-c: summary only)\n"
         "  lsdt crash <cmd> [args]        run program, report crash details");
}

int main(int argc, char **argv)
{
    if (argc < 2) { usage(); return 1; }
    setvbuf(stdout, NULL, _IOLBF, 0);      /* keep tracer/target output in order */
    const char *c = argv[1];
    if (!strcmp(c, "sys"))     return cmd_sys();
    if (!strcmp(c, "info")    && argc >= 3) return cmd_info(atoi(argv[2]));
    if (!strcmp(c, "fds")     && argc >= 3) return cmd_fds(atoi(argv[2]));
    if (!strcmp(c, "threads") && argc >= 3) return cmd_threads(atoi(argv[2]));
    if (!strcmp(c, "mem")     && argc >= 3) return cmd_mem(atoi(argv[2]));
    if (!strcmp(c, "watch")   && argc >= 3)
        return cmd_watch(atoi(argv[2]), argc > 3 ? atoi(argv[3]) : 500,
                         argc > 4 ? atoi(argv[4]) : 6);
    if (!strcmp(c, "trace")   && argc >= 3) {
        if (!strcmp(argv[2], "-c") && argc >= 4) return cmd_trace(argv + 3, 1);
        return cmd_trace(argv + 2, 0);
    }
    if (!strcmp(c, "crash")   && argc >= 3) return cmd_crash(argv + 2);
    usage();
    return 1;
}
```

6.3.9 Test programs

Three small programs were written to generate the situations that the tool diagnoses. Each is compiled separately by the Makefile.

crash_segv.c: NULL-pointer dereference (SIGSEGV)

```
/* Test program 1: NULL-pointer dereference -> SIGSEGV */
#include <stdio.h>
int main(void)
{
    int *p = NULL;
    printf("about to dereference NULL...\n");
    fflush(stdout);
    *p = 42;
    return 0;
}
```

crash_div.c: integer division by zero (SIGFPE)

```
/* Test program 2: integer divide by zero -> SIGFPE */
#include <stdio.h>
#include <stdlib.h>
int main(int argc, char **argv)
{
    int d = argc - 1;                /* 0 when no argument is given */
    printf("about to divide by %d...\n", d);
    fflush(stdout);
    volatile int x = 100 / d;
    printf("%d\n", x);
    return 0;
}
```

busy_threads.c: three CPU-bound threads that grow memory and keep one file open

```
/* Test program 3: 3 worker threads burning CPU, growing heap, holding a file open */
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <pthread.h>
#include <unistd.h>
static void *worker(void *a)
{
    (void)a;
    char *buf = NULL; size_t sz = 0;
    for (int i = 0; i < 40; i++) {
        sz += 512 * 1024;
        buf = realloc(buf, sz);
        memset(buf, i, sz);
        volatile double x = 0;
        for (long j = 0; j < 20000000; j++) x += j * 0.5;
        usleep(20000);
    }
    return buf;
}
int main(void)
{
    FILE *f = fopen("/tmp/busy_test.txt", "w");
    fputs("lsdt test file\n", f); fflush(f);
    pthread_t t[3];
    for (int i = 0; i < 3; i++) pthread_create(&t[i], NULL, worker, NULL);
    for (int i = 0; i < 3; i++) pthread_join(t[i], NULL);
    fclose(f);
    return 0;
}
```

6.4 Linux System Calls / APIs Used

Chapter 4 listed the system calls; this section explains how the more important ones are used and what pitfalls must be handled.

6.4.1 Reading /proc/PID/stat correctly

The `stat` file is a single line of more than fifty space-separated fields. The second field, the command name, is placed in parentheses and may contain spaces, so parsing starts after the last `)`. The fields that LSDT uses are numbered from 1 in the manual page `proc(5)`:

Field no.	Name	Meaning	Used for
3	state	One-letter scheduling state (R, S, D, T, Z ...)	Thread inspector
14	utime	CPU time in user mode, in clock ticks	CPU %, thread inspector
15	stime	CPU time in kernel mode, in clock ticks	CPU %, thread inspector
20	num_threads	Number of threads in the process	Live monitor
24	rss	Resident set size in pages	Live monitor (x page size / 1024 = KB)

The `sscanf` format in `read_proc_stat` skips the unwanted fields with the assignment-suppression modifier `%*`: five `%*d` (fields 4-8), five `%*u` (9-13), the two wanted values (14-15), four `%*d` (16-19), the wanted thread count (20), then `%*d %*u %*u` (21-23) and finally the wanted RSS (24).

6.4.2 ptrace: reading the tracee's memory

A debugger cannot simply dereference a pointer that belongs to another process. `ptrace(PTRACE_PEEKDATA, pid, addr, 0)` returns one 8-byte word from the tracee's address space. `peek_string` calls it repeatedly with increasing addresses, extracts the bytes of each word (little-endian order) and stops at the NUL terminator, at an error (`errno` set) or when the output buffer is full, so a malicious or corrupted pointer cannot cause an overflow.

6.4.3 Syscall numbers decoded by the tracer (x86_64)

Nr	Name	Nr	Name	Nr	Name	Nr	Name
0	read	9	mmap	59	execve	228	clock_gettime
1	write	10	mprotect	60	exit	231	exit_group
2	open	11	munmap	61	wait4	257	openat
3	close	12	brk	62	kill	262	newfstatat
5	fstat	17	pread64	63	uname	302	prlimit64
21	access	35	nanosleep	158	arch_prctl	318	getrandom
39	getpid	56	clone	202	futex	332	statx

6.4.4 Error handling policy

- Every `open`, `read`, `readlink`, `opendir`, `fork` and `pipe` is checked; errors are reported with `perror` and a non-zero exit code.
- A negative `rax` between -4095 and -1 at syscall exit is an error code of the traced program, not of LSDT, and is reported as such.
- `waitpid` results are always examined with the `W*` macros; the tool never assumes that a child is still alive.

6.5 Process / Thread Implementation

6.5.1 Process model of the tracing commands

For `trace` and `crash` the tool runs as **two processes**: the LSDT parent (the tracer) and the child (the tracee). The child is created by `fork()`. Immediately after `fork` the child is an exact copy of LSDT; it calls `PTRACE_TRACEME`, stops itself with `raise(SIGSTOP)` and then `execvp()` replaces its entire memory image with the target program. The stop before `exec` is important: it guarantees that the parent has a chance to configure the tracing options before the target program executes its first instruction. When `execvp` succeeds the kernel sends the tracee a SIGTRAP, which the tracer simply passes over.

The life cycle of the child is therefore: **created (fork) -> traced (TRACEME) -> stopped (SIGSTOP) -> image replaced (exec) -> running / stopped alternately under PTRACE_SYSCALL or PTRACE_CONT -> terminated (exit or signal) -> reaped (waitpid)**. Because the parent always calls `waitpid` for the child, no zombie process can remain, including in the crash case where the parent explicitly kills the child and then waits for it.

6.5.2 Thread model of the live monitor

The `watch` command is a single process containing **three threads**: the main thread (which only creates and joins the other two), the sampler thread and the logger thread. They share the process's address space, which is why the shared `struct monitor` can be passed by pointer. In `/proc` they appear as three entries in `/proc/PID/task`; the thread inspector can show this if run against LSDT itself while a `watch` is active.

6.5.3 Thread model of the observed program

The test program `busy_threads` creates three worker threads. The thread inspector (Figure 7.6) shows four entries for it: the main thread with TID equal to the PID, which is sleeping in `pthread_join` (state S), and three workers with consecutive TIDs in state R (running). All four share the same PID in the process inspector but have different TIDs, demonstrating the TGID / TID distinction of Section 2.1.3.

6.5.4 Scheduling observations

The CPU utilisation measured by `watch` for the three CPU-bound workers is close to 100%, not 300%, because the machine used for the experiment has a **single virtual CPU** (`nproc` reports 1): the scheduler time-slices the three threads on one core and the process total can never exceed 100% of one CPU. On a multi-core machine the same program would show values up to 300%. The thread inspector shows nearly equal user times for the three workers (for example 0.21 s, 0.22 s and 0.21 s), which demonstrates the fairness of the Completely Fair Scheduler.

6.6 IPC / Synchronization Implementation

The live monitor is a complete, small example of the **producer-consumer problem with a bounded buffer**. Figure 6.1 shows the components.

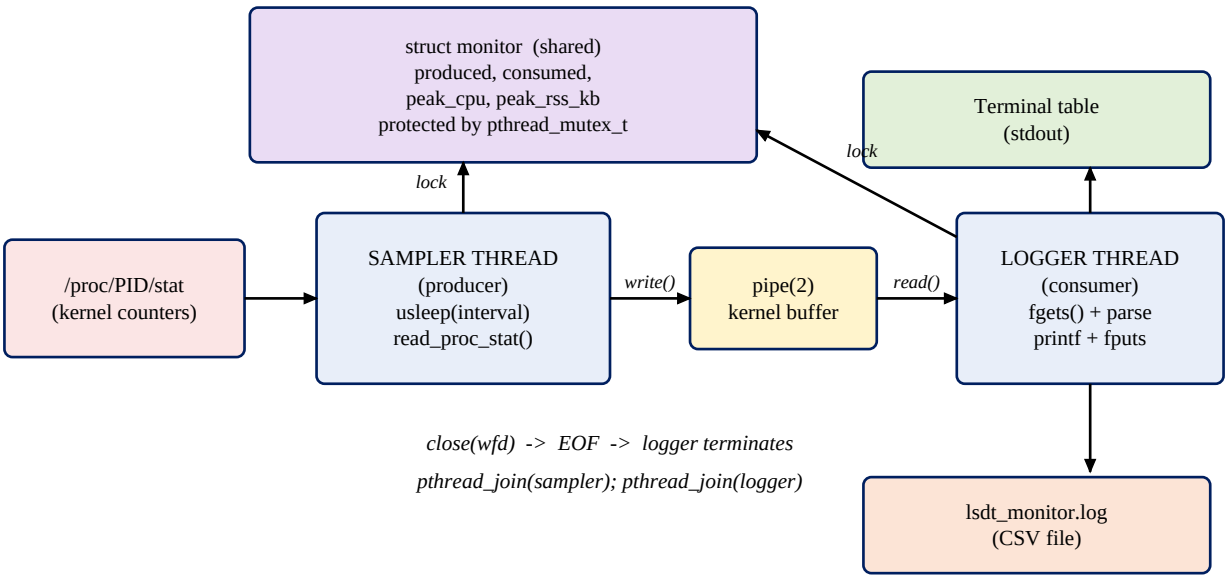


Figure 6.1: Producer-consumer structure of the live resource monitor

6.6.1 Why a pipe is used

The pipe is a kernel-managed bounded buffer (64 KB by default on Linux). When the buffer is empty, a `read` blocks the consumer; when it is full, a `write` blocks the producer. Therefore the two classical synchronisation conditions, "consumer must wait if the buffer is empty" and "producer must wait if the buffer is full", are satisfied without any additional semaphore or condition variable. Bytes are delivered in the order in which they were written, so the log file is always in sample order. Each record is far smaller than `PIPE_BUF` (4096 bytes), so POSIX guarantees that each `write` is atomic, and a record can never be split or interleaved.

6.6.2 Why a mutex is still needed

The four fields `produced`, `consumed`, `peak_cpu` and `peak_rss_kb` are shared between threads. Updating the peak value requires reading it, comparing and writing it, which is not atomic. Without mutual exclusion two threads could read the same old value and one update would be lost (a **race condition**). The mutex `m->lock` is locked around every such group of operations. The critical sections contain only a few arithmetic instructions and **no blocking call** (the `printf`, `write` and `fgets` calls are made outside the lock), so a thread never waits long for the lock.

6.6.3 Absence of deadlock and starvation

- There is **only one lock**, and a thread never tries to acquire it while holding it; therefore a circular wait (the necessary condition for deadlock) cannot occur.
- No lock is held while a thread blocks in a system call.
- Both threads terminate deterministically: the sampler makes exactly `n` iterations and then closes the write end; the logger terminates when it reads end-of-file.

6.6.4 Correct shutdown and data visibility

The write end of the pipe is closed by the sampler thread after the last sample. Once the buffer has been drained, `fgets` in the logger returns `NULL` and the logger exits its loop, closes the log file and returns. `main` calls `pthread_join` on both threads; `pthread_join` provides the memory synchronisation that makes the final values of the shared counters visible to the main thread, so the summary line can be printed without taking the lock. The summary shows `produced == consumed`, which is the check that no record was lost.

6.6.5 IPC between tracer and tracee

The relationship between the tracer and the traced program is itself a form of inter-process communication through the kernel: **ptrace requests** (resume, read registers, read memory) flow from the tracer to the kernel, and **wait notifications** (stop events) flow back through `waitpid`. Signals complete the picture: the SIGSTOP raised by the child, the SIGTRAP sent at `exec`, and the SIGKILL sent by the crash analyser are all asynchronous messages exchanged through the kernel's signal subsystem.

6.7 Memory Management Implementation

Memory management appears in the project in two ways: how the tool manages its **own** memory, and how it reports the memory of **other** processes.

6.7.1 The tool's own memory

LSDT contains **no call to** `malloc`**,** `calloc`**,** `realloc` **or** `free`. All storage is on the stack (local arrays) or static, and its size is fixed at compile time. This design has important consequences:

- **No memory leaks:** there is nothing to forget to free.
- **No heap corruption:** there is no use-after-free or double free.
- **Bounded use:** the largest buffers are 8 KB (the status file) and the 512-entry syscall table; total stack use of any function is well below the default 8 MB stack limit.
- **Predictable behaviour for a debugger:** the tool adds almost no memory pressure to the system it observes.

Function	Buffers	Bounds control
`cmd_info`	`path[64]`, `buf[8192]`, `v[256]`	`read_file` writes at most `n-1` bytes; `status_field` copies at most `n-1` bytes
`cmd_trace`	`struct sysstat tab[512]`, `path[256]`, `line[512]`	The syscall number is checked against `MAXSYS` before indexing; `peek_string` and `snprintf` take the buffer size
`cmd_mem`	`line[512]`, `name[300]`	`fgets` with `sizeof`; `sscanf` with the field width `%299`
`cmd_fds`, `cmd_threads`	`p[512]`, `target[512]`, `buf[1024]`	`readlink` size is `sizeof - 1`; the NUL is added manually
`sampler_thread`	`rec[128]`	`snprintf` returns the length actually used and `write` sends exactly that length

6.7.2 Resource clean-up

Every resource acquired is released on every path: files opened with `open` are closed in `read_file`; `opendir` is paired with `closedir`; `fopen` with `fclose`; the read end of the pipe is wrapped by `fdopen` and is released by the `fclose` in the logger, and the write end is closed by the sampler. The child process is always reaped with `waitpid`.

6.7.3 Reporting other processes' memory

The memory analyser reports what the kernel's virtual-memory subsystem has recorded. It distinguishes **virtual** from **resident** memory: the sum of anonymous mappings (about 227 MB for the test program) is mostly virtual address space reserved for the stacks and arenas of the three threads, each of which has a 64 MB `---p` region with no access permission that is mapped but never touched; only the pages that have been used are counted in Rss (about 7.9 MB at the moment of the measurement). The `Pss` line is lower than `Rss` because it divides the shared library pages among all processes that map them. The `Private_Dirty` line identifies the memory that belongs only to this process and has been modified, which is the part that would have to be written to swap or lost on exit.

6.7.4 Reading the tracee's memory safely

Direct memory access to another process is possible only through ptrace. The tool reads **one word at a time** and validates every step, because the target controls the pointer values: a null or invalid file-name pointer simply makes `PEEKDATA` fail with `EFAULT` and the string is shown as empty. A tracee cannot, therefore, crash the tracer by passing a bad pointer.

6.8 Optional User Interface Implementation

The main interface of LSDT is the command line, designed for clarity and for use in scripts. Three optional interface features were added to make the tool convenient for demonstration.

6.8.1 Self-describing command-line interface

Running `./lsdt` without arguments, or with an unknown command, prints the help text shown in Figure 7.3. Each module prints a title line that identifies the module and the PID, then labelled values aligned in columns with `printf` field widths (`%-28s`, `%9lu`, `%12.6f`), so that columns line up without any external formatting tool.

6.8.2 Demonstration script

The script `demo.sh` builds the tool and runs every module against the test programs in a fixed order. It is used for lab demonstrations and for regression testing after any change to the source.

```
#!/bin/bash
# demo.sh - runs every LSDT module against the supplied test programs
make -s
./lsdt sys
./busy_threads & PID=$!
sleep 1
./lsdt info $PID
./lsdt threads $PID
./lsdt fds $PID
./lsdt mem $PID
./lsdt watch $PID 400 5
kill $PID 2>/dev/null; wait $PID 2>/dev/null
./lsdt trace -c /bin/ls demo_dir
./lsdt crash ./crash_segv ; true
./lsdt crash ./crash_div  ; true
```

6.8.3 Machine-readable log

The `watch` command writes `lsdt_monitor.log` in CSV form. This is the interface to external graphical tools: the chart in Figure 7.16 was drawn from the data of such a run with a few lines of Python, and the file can equally be opened in a spreadsheet.

6.8.4 Possible terminal user interface (future work)

A full-screen interface similar to `htop` could be added with the `ncurses` library, calling the existing module functions on a timer and re-drawing the screen. The planned layout is sketched below; it would need no change to the data-collection code.

```
+-- LSDT ------------------------------------------------------+
| PID 351  busy_threads   S   threads 4   RSS 28 MB   CPU 99 %   |
+-- [F1] Info [F2] FDs [F3] Threads [F4] Memory [F5] Trace -------+
| TID    NAME           STATE   UTIME   STIME                     |
| 351    busy_threads   S       0.00    0.00                      |
| 353    busy_threads   R       0.21    0.00                      |
| 354    busy_threads   R       0.22    0.00                      |
+-- q: quit   +/-: interval   s: save log -------------------------+
```

CHAPTER - 7
EXECUTION AND DEMONSTRATION

All screenshots in this chapter are the **real output** of the program, produced by running the commands in Section 7.2 on Ubuntu 24.04 LTS (x86_64, GCC 13.3.0, one virtual CPU, 4 GB RAM). The terminal windows were rendered directly from the captured text so that no value has been edited. Process IDs, addresses and timings differ from run to run, as expected (the executables are position-independent and address-space layout randomisation is active).

7.1 Compilation in Ubuntu

Open a terminal in Ubuntu (or in the WSL console), change to the project directory and compile with GNU Make, which builds the tool and the three test programs:

```
$ cd ~/lsdt
$ make
```

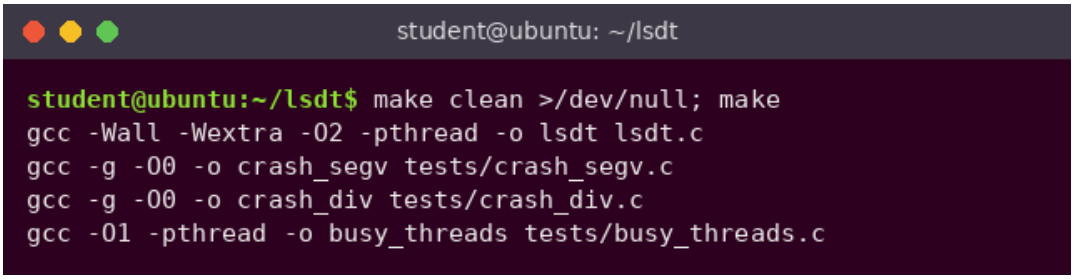


Figure 7.1: Building the tool and the test programs with `make` *(no warnings)*

The same result can be obtained without make by compiling the tool directly. The flag `-pthread` is required because of the live monitor, and `-Wall -Wextra` enables the compiler's warnings; the build prints nothing but the size of the produced executable, which confirms that there are **no warnings or errors**.

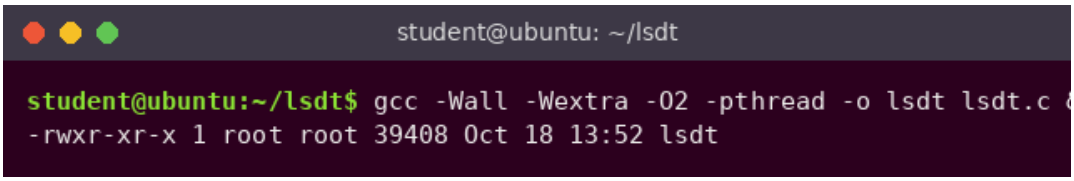


Figure 7.2: Direct compilation with gcc and the size of the executable

7.2 Execution Commands

Purpose	Command
Show help	`./lsdt`
System overview	`./lsdt sys`
Start a test target in the background	`./busy_threads &` (the shell prints its PID; use `$!` for the last background PID)
Inspect a process	`./lsdt info <PID>`
Inspect its threads	`./lsdt threads <PID>`
List open file descriptors	`./lsdt fds <PID>`
Analyse the memory map	`./lsdt mem <PID>`
Monitor for 6 samples, 400 ms apart	`./lsdt watch <PID> 400 6`
Trace all system calls of a command	`./lsdt trace /bin/cat demo_dir/sample.txt`
Statistics only	`./lsdt trace -c /bin/ls demo_dir`
Trace a failing call	`./lsdt trace /bin/cat /no/such/file`
Analyse a NULL-pointer crash	`./lsdt crash ./crash_segv`
Analyse a divide-by-zero crash	`./lsdt crash ./crash_div`
Check a healthy program	`./lsdt crash /bin/true`
Run everything automatically	`./demo.sh`
Clean the build	`make clean`

7.3 Sample Inputs and Outputs

Table 7.1 lists the test cases that were executed, with the input, the expected behaviour and the observed result. All 14 tests passed.

ID	Module	Input	Expected	Observed	Result
T1	help	`./lsdt`	Usage text; exit 1	Usage text with 8 commands	Pass
T2	sys	`./lsdt sys`	Load, uptime, memory, process count	MemTotal 4093768 kB; 52 processes	Pass
T3	info	`info <PID>` of the test program	Name, 4 threads, memory, paths	busy_threads, Threads 4, VmRSS 7896 kB	Pass
T4	threads	`threads <PID>`	4 entries (main + 3 workers)	4 entries; workers in state R	Pass
T5	fds	`fds <PID>`	fd 0, 1, 2 and the open file	4 descriptors incl. /tmp/busy_test.txt	Pass
T6	mem	`mem <PID>`	Heap, stack, libc, guard pages	39 mappings; 132 KB heap, 132 KB stack, libc found	Pass
T7	watch	`watch <PID> 400 6`	6 samples; produced = consumed	produced = 6, consumed = 6	Pass
T8	trace	`trace /bin/cat demo_dir/sample.txt`	openat of the file; output text visible	openat("demo_dir/sample.txt") = 3 and text printed in correct order	Pass
T9	trace -c	`trace -c /bin/ls demo_dir`	Statistics table with total	TOTAL 76 calls; getdents64 present	Pass
T10	trace (error)	`trace /bin/cat /no/such/file`	openat returns ENOENT; exit status 1	`= -1 ENOENT (No such file or directory)`; exited with 1	Pass

ID	Module	Input	Expected	Observed	Result
T11	crash (SEGV)	`crash ./crash_segv`	SIGSEGV, SEGV_MAPERR, address 0	SIGSEGV, SEGV_MAPERR, fault address (nil), region not mapped	Pass
T12	crash (FPE)	`crash ./crash_div`	SIGFPE, integer divide by zero	SIGFPE, FPE_INTDIV	Pass
T13	crash (healthy)	`crash /bin/true`	No crash reported	"exited normally with status 0"	Pass
T14	bad PID	`info 999999`	Error message, exit 1	`status: No such file or directory`	Pass

Table 7.1: Test cases and results

7.3.1 How to read the outputs

Output	Field	Meaning
info	`State S (sleeping)`	The main thread is blocked in `pthread_join`, waiting for the workers
info	`VmSize` vs `VmRSS`	Virtual size (230040 kB) is much larger than resident size (7896 kB); most reserved address space has never been touched (demand paging)
mem	`---p` with 65404 KB	Guard / reserved regions of the thread arenas: mapped but with no permission
mem	`Pss` < `Rss`	Shared library pages are counted fractionally in Pss
watch	CPU(%) near 100	Three CPU-bound threads on a one-CPU machine saturate the processor
watch	RSS(KB) rising	The workers keep reallocating and touching larger buffers: memory growth over time
trace	`<0.000007>`	Seconds spent between syscall entry and exit
trace	`= -1 ENOENT`	The kernel returned error number 2: the file does not exist
crash	SEGV_MAPERR	The faulting address is not inside any mapping
crash	RAX 0x64 (div test)	100 decimal: the dividend loaded in RAX just before the `idiv` instruction

7.3.2 Performance: overhead of tracing

Tracing stops the program twice per system call, so a traced program runs slower. To quantify this, the command `find /usr/lib -maxdepth 3` (which makes 3,364 system calls) was run 15 times natively and 15 times under `lsdt trace -c`, and the median wall-clock times were compared (Table 7.2).

Measurement	Native	Under LSDT trace -c	Ratio
Median run time (15 runs)	4.46 ms	28.33 ms	6.4 x slower
System calls intercepted	-	3,364	-
Extra time per system call	-	about 7 microseconds	-

Table 7.2: Tracing overhead measured on the test machine (1 vCPU)

A factor of about six is typical for ptrace-based tracers, because each stop requires two context switches between tracee and tracer. For the short diagnostic runs for which the tool is intended, the absolute cost is only milliseconds. The `watch`, `info`, `mem`, `fds` and `threads` commands do not stop the target at all and therefore add no measurable overhead to it.

7.4 Screenshots of Execution

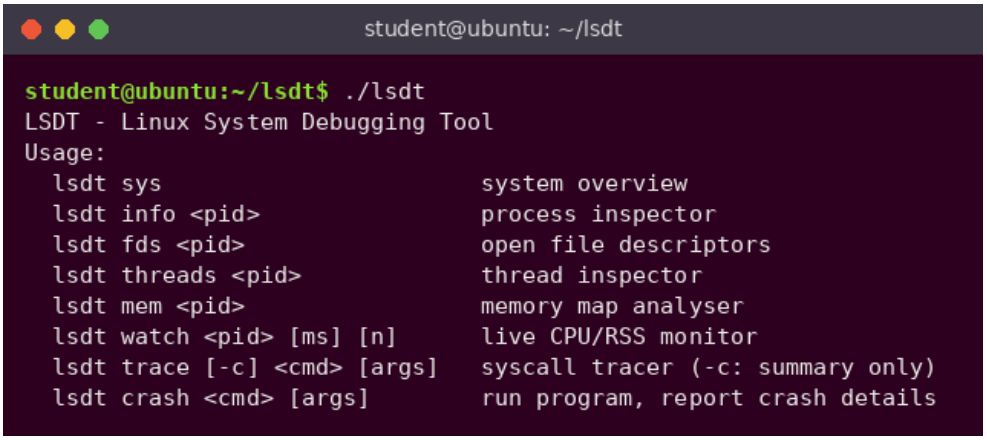


Figure 7.3: Help screen printed by `./lsdt` *with no arguments*

The help screen lists every command with its arguments.

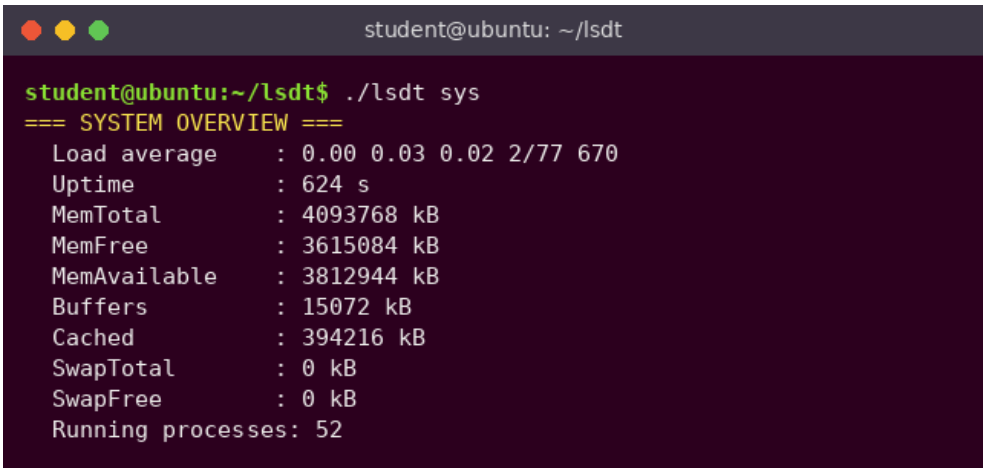


Figure 7.4: `lsdt sys` *- system overview*

Load average, uptime, memory totals and the number of processes give context for the other measurements.

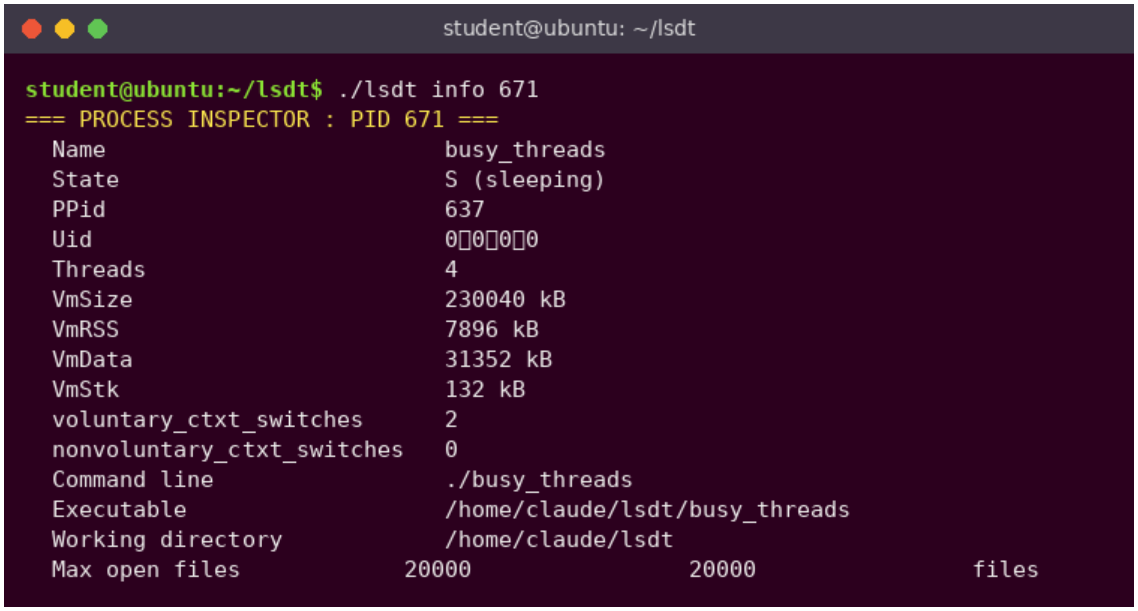


Figure 7.5: `lsdt info <PID>` *- process inspector*

The test program has four threads, a resident size far below its virtual size, and a state of "sleeping" because the main thread waits for its workers.

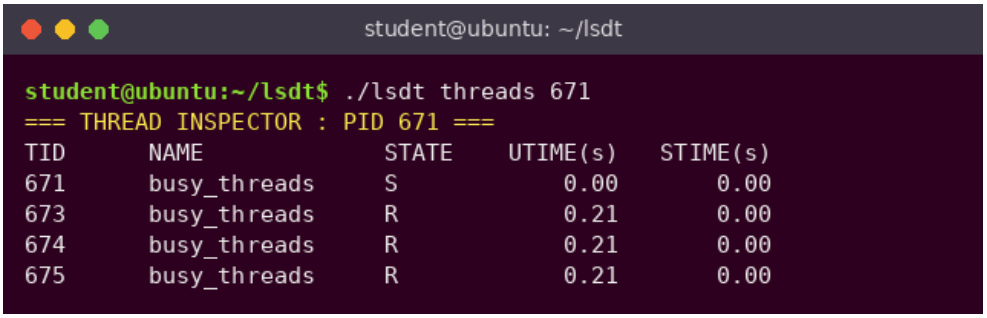


Figure 7.6: `lsdt threads <PID>` - *thread inspector*

One main thread (TID = PID) and three worker threads (consecutive TIDs) that are in state R and have consumed almost identical CPU time.

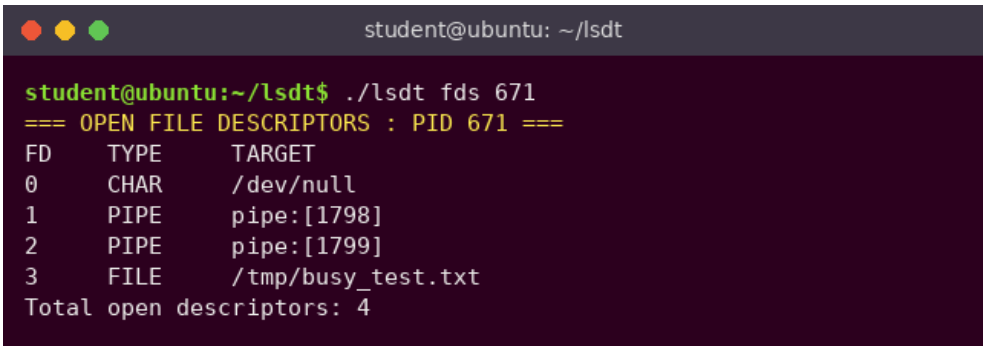


Figure 7.7: `lsdt fds <PID>` - *open file descriptors*

Descriptors 0 to 2 are the standard streams (here a device and two pipes because the program runs under a script); descriptor 3 is the file the test program opened.

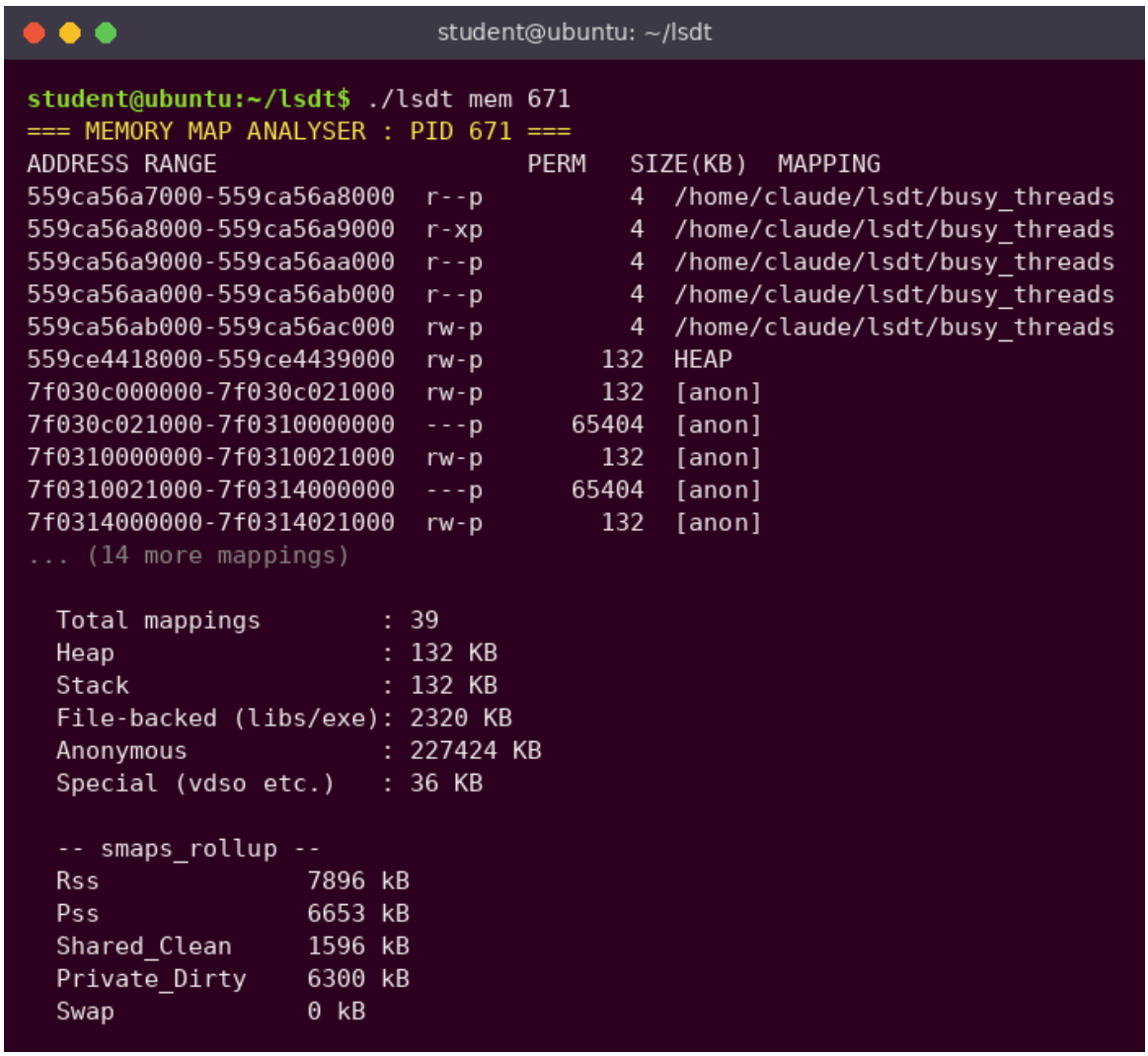


Figure 7.8: `lsdt mem <PID>` - *memory map analysis*

The report shows the executable, the heap, the stack, libc, per-thread arenas (8 MB rw-p) with their guard pages (4 KB ---p), and the totals and smaps_rollup summary.

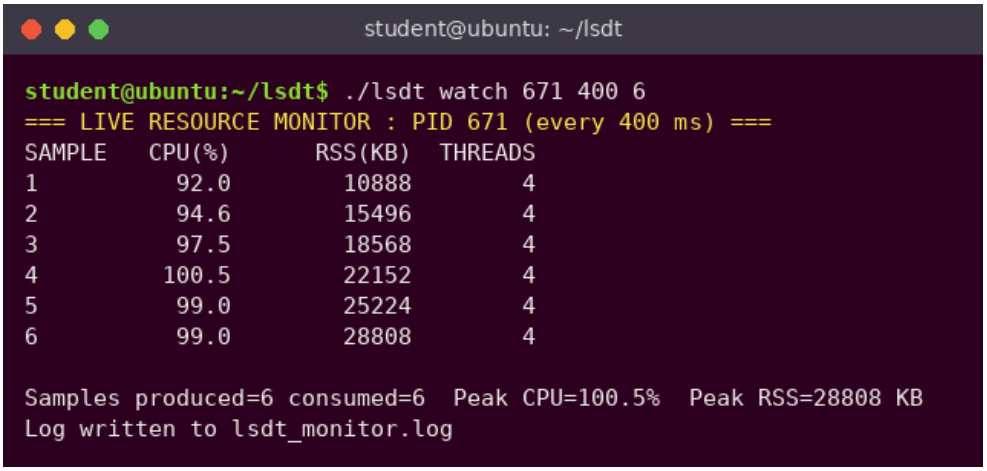


Figure 7.9: `lsdt watch <PID> 400 6` - *live resource monitor*

Six samples show CPU close to 100% and steadily increasing resident memory; the summary confirms that all produced records were consumed.

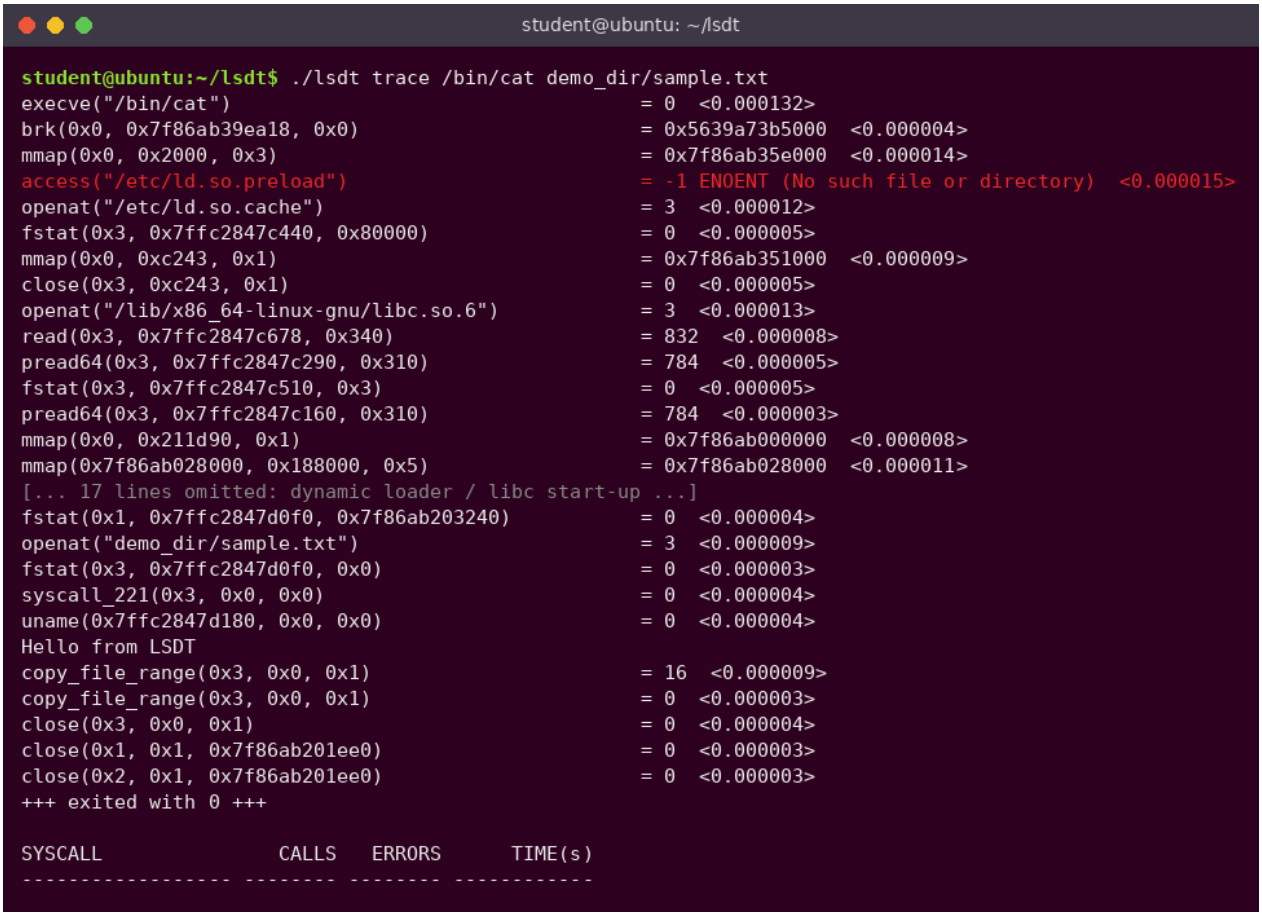


Figure 7.10: `lsdt trace /bin/cat demo_dir/sample.txt` - *system-call trace (middle part omitted)*

The trace shows how `cat` starts: the dynamic loader opens libc, maps it, sets up thread-local storage, and finally the `openat` of the file and the `copy_file_range` calls that copy 16 bytes to standard output. The text "Hello from LSDT" appears in the right position between the calls because of the line-buffering fix.

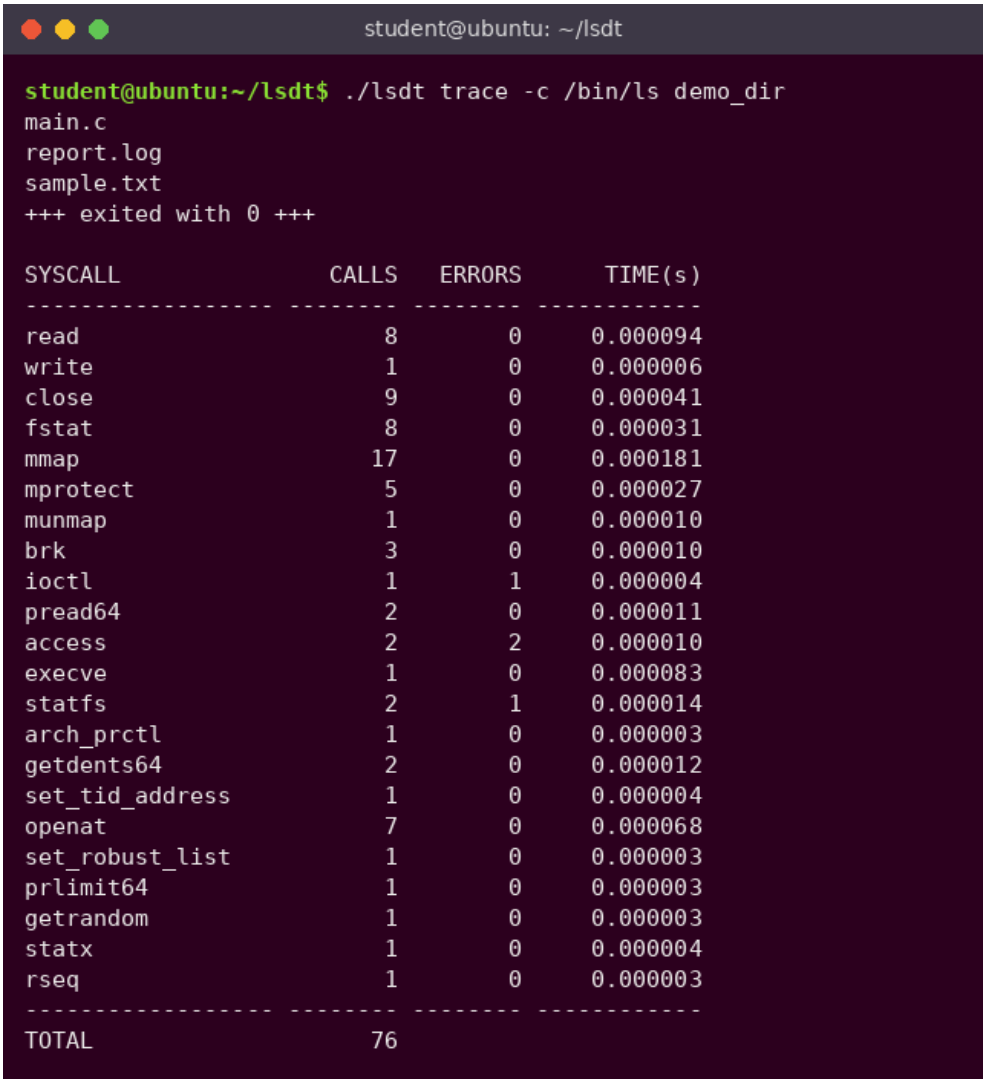


Figure 7.11: `lsdt trace -c /bin/ls demo_dir` - *statistics mode*

Without the per-call lines the tool prints the target's output and a table: 76 system calls in total, including two `getdents64` calls to read the directory; the ERRORS column shows a few failed probe calls (`access`, `ioctl`, `statfs`), which are normal.

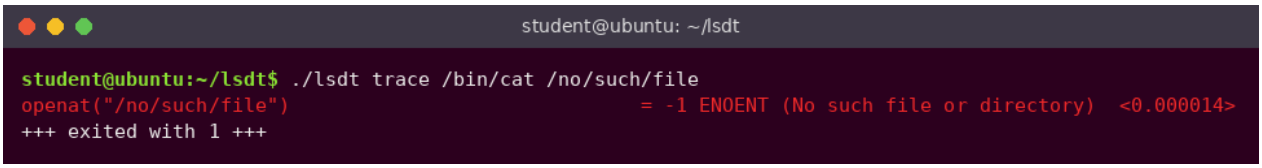


Figure 7.12: Tracing a failing call

The failing call is visible instantly: the kernel returned ENOENT for the file name, and the process exited with status 1.

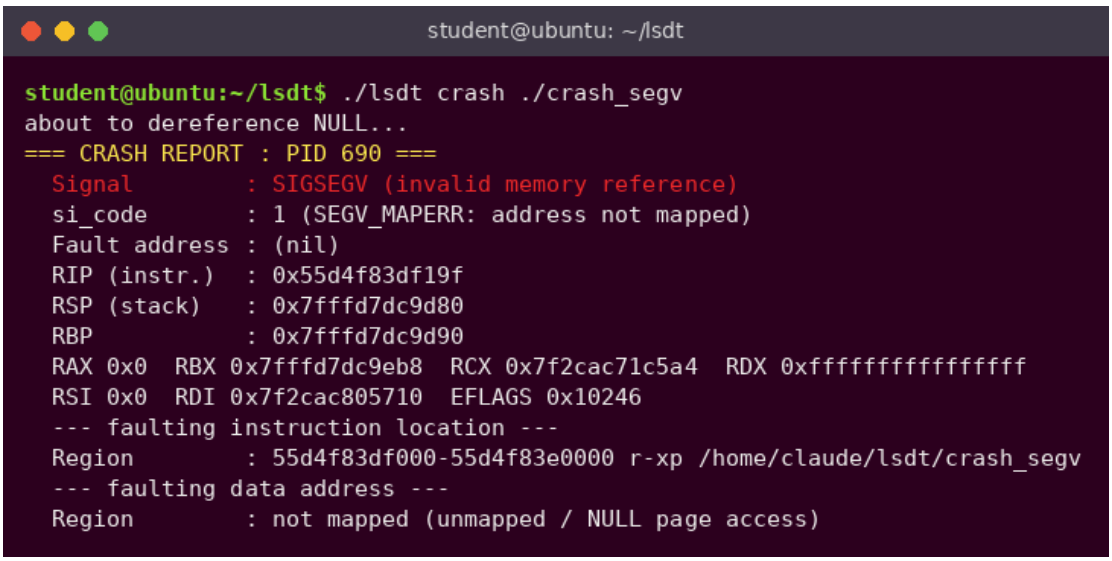


Figure 7.13: `lsdt crash ./crash_segv` - *NULL-pointer dereference*

The report identifies SIGSEGV with cause SEGV_MAPERR at address (nil): a NULL pointer was dereferenced. The instruction pointer is inside the executable's code region; the data address is not mapped at all.

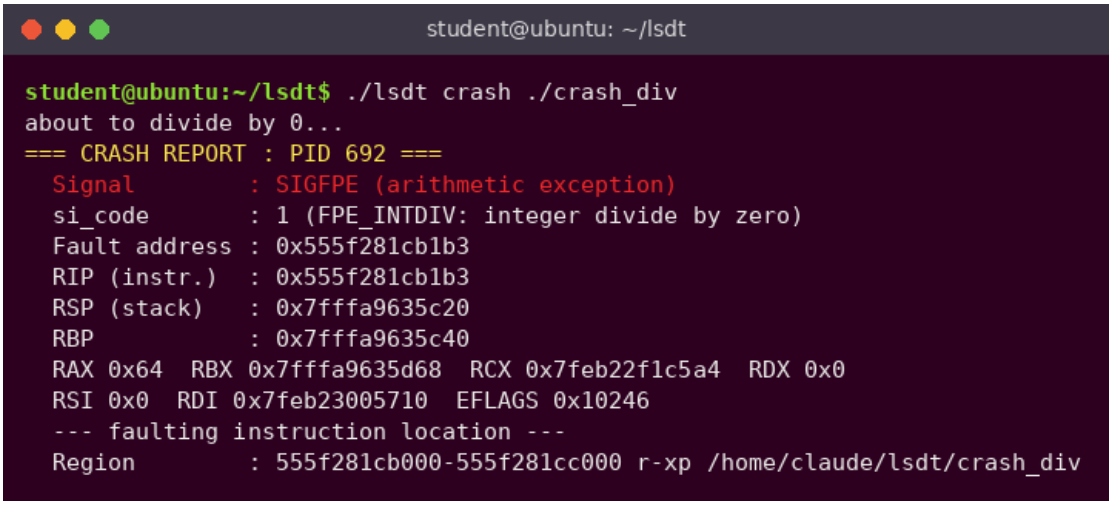


Figure 7.14: `lsdt crash ./crash_div` - *integer divide by zero*

SIGFPE with code FPE_INTDIV. RAX contains 0x64 (100, the dividend) and the divisor in memory is zero.

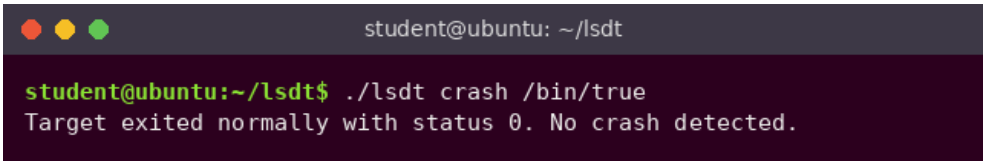


Figure 7.15: `lsdt crash /bin/true` - *healthy program*

A program that terminates normally produces no crash report.

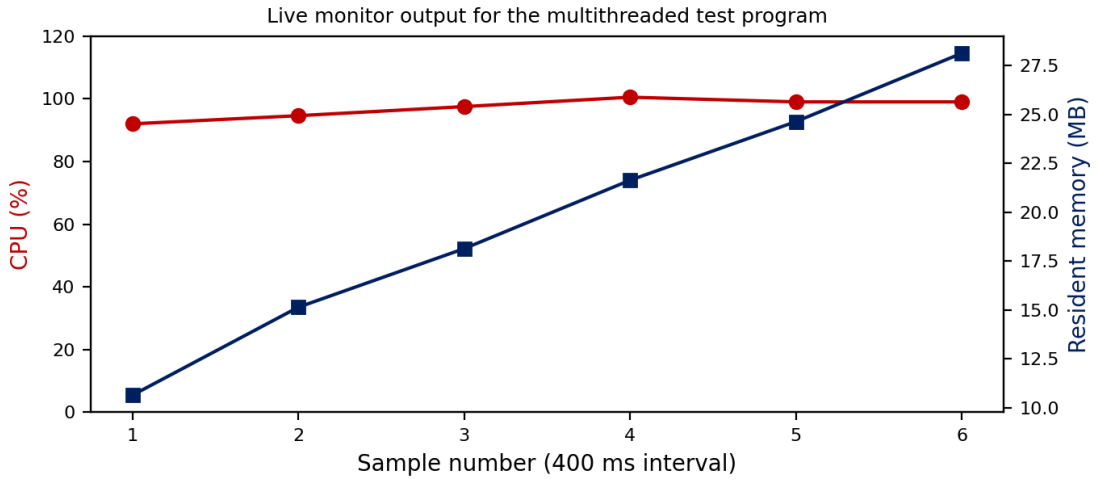


Figure 7.16: CPU and resident memory from the live monitor log (plotted with matplotlib)

The chart is drawn from the six samples of Figure 7.9: CPU stays near the 100% ceiling of the single CPU while resident memory grows linearly, which is the signature of a program that continuously allocates and touches memory.

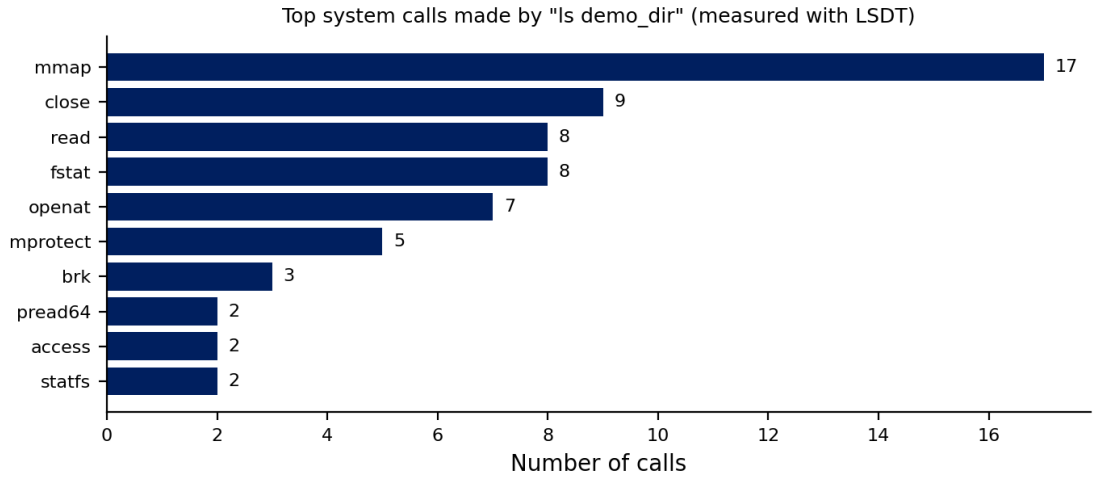


Figure 7.17: The ten most frequent system calls of `ls demo_dir` *as measured by LSDT*

Memory-management calls (`mmap`, `mprotect`, `brk`) and file calls (`openat`, `read`, `close`, `fstat`) dominate even for a trivial command, because most of the work is the dynamic loader mapping the shared libraries.

7.5 Demonstration of OS Concepts

The table maps each Operating System concept of the course to the LSDT command that demonstrates it and to the evidence in the outputs above.

OS concept	Demonstrated by	Evidence
System-call interface, user / kernel mode	`trace`	Every library call ends in a kernel call; the trace of `cat` shows `openat`, `read`, `copy_file_range`, `close`; durations show time spent in the kernel
Process creation: fork-exec-wait	`trace`, `crash`	The tool forks, the child execs the target; the parent waits and obtains the exit status (+++ exited with 0 +++)

OS concept	Demonstrated by	Evidence
Process states and PCB	`info`	State S (sleeping), PPid, UID, context-switch counters read from the kernel's process descriptor
Threads (TGID / TID)	`threads`, `info`	One PID, four TIDs; workers in state R
Virtual memory, demand paging	`mem`, `info`	VmSize of about 230 MB versus VmRSS of about 8 MB; guard pages with no permissions
Memory protection	`mem`, `crash`	Permissions r-xp / rw-p / ---p; access to an unmapped address raises SIGSEGV
Shared memory and copy-on-write	`mem`	Pss lower than Rss because libc pages are shared; Shared_Clean = 1,596 kB
Heap and stack management	`mem`	Separate [heap] and [stack] regions; thread arenas as anonymous mappings
Signals and exceptions	`crash`	CPU exceptions converted to SIGSEGV / SIGFPE with si_code and fault address
File descriptors, VFS	`fds`	Descriptor table with FILE, PIPE and CHAR entries
Scheduling and CPU accounting	`watch`, `threads`	CPU % computed from jiffies; equal user times for equal threads on one CPU
Context switching	`info`	voluntary and non-voluntary context-switch counters
Inter-process communication	`watch`	Pipe between two threads; ptrace/wait between two processes
Synchronisation, producer-consumer, mutex	`watch`	produced = consumed = 6 with a mutex-protected shared structure
/proc pseudo-filesystem	`info`, `mem`, `sys`	All data is read from kernel-generated files

7.5.1 Independent validation of the crash analyser with objdump

To confirm that the instruction pointer reported by LSDT is genuinely the faulting instruction, the test programs were disassembled with `objdump -d`. Because the programs are position-independent executables, the run-time address equals a load base (a multiple of 4096) plus the file offset, so the **low 12 bits of RIP must equal the low 12 bits of the offset** of the faulting instruction.

Test	RIP reported by LSDT	Offset in objdump	Instruction in objdump	Match
crash_segv	0x55d4f83df19f (low bits 0x19f)	0x119f	`movl $0x2a,(%rax)` with RAX = 0: store through a NULL pointer	Yes
crash_div	0x555f281cb1b3 (low bits 0x1b3)	0x11b3	`idivl -0x4(%rbp)` with the divisor variable equal to 0	Yes

In both cases the address reported by the tool is exactly the machine instruction that caused the exception, and the register contents (RAX = 0 for the NULL store; RAX = 0x64, RDX = 0 for the division) are those expected from the disassembly. This confirms the correctness of the crash analyser.

CHAPTER - 8
CONCLUSION

8.1 Summary of the Project

This project designed and implemented the **Linux System Debugging Tool (LSDT)**, a single command-line program written in C that consolidates the most common diagnostic activities of Linux system debugging. Starting from the observation that existing debugging utilities are fragmented and too large to be studied, the team built a compact tool of about 600 lines that obtains all of its information directly from the kernel through the /proc filesystem, the ptrace system call, signals, pipes and POSIX threads.

The tool consists of eight modules: process inspector, file-descriptor inspector, thread inspector, memory-map analyser, live resource monitor, system-call tracer, crash analyser and system overview. They were implemented following a layered architecture with shared helper functions, validated with three purpose-written test programs, and measured on an Ubuntu 24.04 system. The results confirm that the tool correctly reports process state, memory layout, system calls and the exact cause of crashes, and that its output agrees with independent disassembly of the test programs.

Beyond the working software, the project demonstrates how the core topics of an Operating Systems course (system calls, processes and threads, virtual memory, signals, file descriptors, scheduling, inter-process communication and synchronisation) are connected in a real program.

8.2 Objectives Achieved

No.	Objective (Section 1.4)	Status	Evidence
1	Study and apply /proc, ptrace, signals, pipes, pthreads	Achieved	Chapters 2, 4 and 6
2	Modular architecture with shared helpers	Achieved	Figure 5.1, Section 5.1
3	Process, descriptor, thread inspectors and system overview	Achieved	Figures 7.4 to 7.7
4	Memory-map analyser with RSS / PSS / private-dirty	Achieved	Figure 7.8
5	System-call tracer with arguments, results, statistics	Achieved	Figures 7.10 to 7.12
6	Crash analyser with signal, address, registers, region	Achieved	Figures 7.13, 7.14; objdump validation in 7.5.1
7	Live monitor with pipe and mutex	Achieved	Figure 7.9; produced = consumed
8	Under 700 lines, warning-free, no dynamic allocation	Achieved	607 lines; zero warnings with -Wall -Wextra; no malloc
9	Validate with test programs and measure overhead	Achieved	Table 7.1 (14 of 14 passed); Table 7.2 (6.4 x)
10	Document as a teaching aid	Achieved	This report, `demo.sh`, test programs

8.3 Results Achieved

Result	Value
Number of modules / commands	8 modules, 8 commands
Size of the tool	607 lines of C; executable of about 39 KB
Compiler warnings	0 (with `-Wall -Wextra`)
Dynamic memory allocation in the tool	None
External library dependencies	None except glibc / pthreads
Test cases passed	14 of 14
Crash causes correctly identified	NULL-pointer dereference (SIGSEGV / SEGV_MAPERR) and divide-by-zero (SIGFPE / FPE_INTDIV)
Crash address accuracy	RIP matched the objdump offset exactly in both tests
Syscalls decoded for `ls demo_dir`	76 calls with names, errors and timings
Monitoring integrity	Records produced = records consumed (6 / 6)
Tracing overhead	About 6.4 x on a syscall-intensive command (about 7 microseconds per call)

8.4 Skills and Learning Outcomes

Through this project the team members acquired the following knowledge and skills:

- **Operating-system internals:** understanding of the process control block, process states, threads (TGID / TID), virtual address-space layout, demand paging, RSS / PSS, file-descriptor tables and the system-call interface.
- **Linux system programming in C:** use of `fork`, `exec`, `wait`, `ptrace`, `pipe`, `readlink`, `opendir`, `clock_gettime` and the signal and wait-status macros.
- **Debugging techniques:** reading registers, interpreting `si_code` and fault addresses, disassembling code with `objdump`, and relating instruction pointers to program behaviour.
- **Concurrent programming:** implementing a producer-consumer pair with `pthread_create`, a pipe and a mutex, and reasoning about race conditions, deadlock and termination.
- **Defensive programming:** bounded buffers, return-value checking, error reporting and resource clean-up without dynamic allocation.
- **Performance measurement:** designing a repeatable experiment to measure tracing overhead.
- **Tooling and teamwork:** GCC warnings and flags, Makefiles, Git, WSL / Ubuntu set-up, technical documentation and division of work among three members.

8.5 Limitations and Future Scope

8.5.1 Limitations

- The register layout and system-call numbers are for **x86_64** only; ARM64 would need a different table and register structure.
- The tracer follows only the initial process; threads and child processes created by the target (`PTRACE_O_TRACECLONE`, `TRACEFORK`) are not traced.

- System-call arguments are shown as raw hexadecimal values except file names; structures and flags are not decoded.
- Only about forty system-call names are known; the others are shown as `syscall_N` (but are still counted correctly).
- No symbol resolution: the crash report shows addresses and memory regions, not function names or source lines.
- Inspecting other users' processes requires administrator privileges.
- Tracing slows the target by roughly a factor of six.

8.5.2 Future enhancements

- **Complete syscall table** generated from the kernel headers, with argument decoding (flags, structures, strings).
- **Multi-process and multi-thread tracing** using `PTRACE_O_TRACEFORK`, `TRACECLONE` and `TRACEEXEC`; attach to a running process with `PTRACE_ATTACH` / `PTRACE_SEIZE`.
- **Symbolic crash reports:** read the ELF symbol table and DWARF line information, or call `addr2line`, to display function names and source lines; add a stack back-trace by walking the frame-pointer chain.
- **Memory-leak detection** by intercepting `brk` / `mmap` / `munmap` or by using an `LD_PRELOAD` library that wraps `malloc` and `free`.
- **Signal and network monitoring:** trace signal delivery and list sockets from `/proc/net`.
- **Terminal UI** with `ncurses` and a web dashboard fed by the CSV log.
- **Alert thresholds** (CPU, memory or descriptor count) with e-mail or log notifications.
- **Container awareness:** report cgroup limits and namespaces from `/proc/PID/cgroup` and `/proc/PID/ns`.
- **Support for ARM64** through architecture-specific tables selected at compile time.

CHAPTER - 9
REFERENCES

9.1 Books

1. A. Silberschatz, P. B. Galvin and G. Gagne, **Operating System Concepts**, 10th ed., Wiley, 2018.
2. A. S. Tanenbaum and H. Bos, **Modern Operating Systems**, 4th ed., Pearson, 2015.
3. M. Kerrisk, **The Linux Programming Interface: A Linux and UNIX System Programming Handbook**, No Starch Press, 2010.
4. W. R. Stevens and S. A. Rago, **Advanced Programming in the UNIX Environment**, 3rd ed., Addison-Wesley, 2013.
5. R. Love, **Linux Kernel Development**, 3rd ed., Addison-Wesley, 2010.
6. D. P. Bovet and M. Cesati, **Understanding the Linux Kernel**, 3rd ed., O'Reilly Media, 2005.
7. R. E. Bryant and D. R. O'Hallaron, **Computer Systems: A Programmer's Perspective**, 3rd ed., Pearson, 2015.
8. B. W. Kernighan and D. M. Ritchie, **The C Programming Language**, 2nd ed., Prentice Hall, 1988.
9. R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, **Operating Systems: Three Easy Pieces**, Arpaci-Dusseau Books, 2018.

9.2 Research Papers

1. D. M. Ritchie and K. Thompson, "The UNIX Time-Sharing System," **Communications of the ACM**, vol. 17, no. 7, pp. 365-375, 1974.
2. T. J. Killian, "Processes as Files," **Proceedings of the USENIX Summer Conference**, Salt Lake City, 1984.
3. N. Nethercote and J. Seward, "Valgrind: A Framework for Heavyweight Dynamic Binary Instrumentation," **Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)**, pp. 89-100, 2007.
4. K. Serebryany, D. Bruening, A. Potapenko and D. Vyukov, "AddressSanitizer: A Fast Address Sanity Checker," **Proceedings of the USENIX Annual Technical Conference**, 2012.
5. M. Desnoyers and M. R. Dagenais, "The LTTng Tracer: A Low Impact Performance and Behavior Monitor for GNU/Linux," **Proceedings of the Ottawa Linux Symposium**, 2006.

9.3 Linux / GNU Documentation

1. Linux man-pages: `ptrace(2)`, `wait(2)`, `fork(2)`, `execve(2)`, `pipe(2)`, `clone(2)`, `syscalls(2)`, `proc(5)`, `signal(7)`, `sigaction(2)`, `pthreads(7)`, `pthread_mutex_lock(3)`, `pipe(7)`, `core(5)`. https://man7.org/linux/man-pages/
2. The Linux kernel documentation: The /proc Filesystem. https://docs.kernel.org/filesystems/proc.html

3. The Linux kernel documentation: Yama Linux Security Module. https://docs.kernel.org/admin-guide/LSM/Yama.html
4. GNU Compiler Collection (GCC) Online Documentation. https://gcc.gnu.org/onlinedocs/
5. The GNU C Library Reference Manual. https://www.gnu.org/software/libc/manual/
6. GNU Make Manual. https://www.gnu.org/software/make/manual/
7. GDB: The GNU Project Debugger, Documentation. https://sourceware.org/gdb/documentation/

9.4 Online Resources

1. strace project home page. https://strace.io/
2. Ubuntu on WSL, Ubuntu documentation. https://ubuntu.com/wsl
3. Microsoft Learn: Windows Subsystem for Linux documentation. https://learn.microsoft.com/windows/wsl/
4. B. Gregg, Linux performance and tracing resources. https://www.brendangregg.com/linuxperf.html
5. Linux x86_64 system-call table (ChromiumOS documentation). https://chromium.googlesource.com/chromiumos/docs/+/master/constants/syscalls.md